The Journal of Systems and Software 219 (2025) 112215

Contents lists available at ScienceDirect

The Journal of Systems & Software

journal homepage: www.elsevier.com/locate/jss

Development and benchmarking of multilingual code clone detector☆

Wenqing Zhu [a,*], Norihiro Yoshida [b], Toshihiro Kamiya [c], Eunjong Choi [d], Hiroaki Takada [a]

[a] Graduate School of Informatics, Nagoya University, Furo-cho, Chikusa-ward, Nagoya-City, 464-8601, Aichi, Japan
[b] Graduate School of Information Science and Engineering, Ritsumeikan University, 2-150 Iwakura-cho, Ibaraki, 567-8570, Osaka, Japan
[c] Interdisciplinary Faculty of Science and Engineering, Shimane University, 1060 Nishikawatsu-cho, Matsue, 690-8504, Shimane, Japan
[d] Faculty of Information and Human Sciences, Kyoto Institute of Technology, Matsugasaki, Sakyo-ku, Kyoto, 606-8585, Kyoto, Japan

ARTICLE INFO

MSC:
68-01

Keywords:
Code clone
Parser generation
Benchmark testing

ABSTRACT

The diversity of programming languages is growing, making the language extensibility of code clone detectors crucial. However, this is challenging for most existing clone detection detectors because the source code handler needs modifications, which requires specialist-level knowledge of the targeted language and is time-consuming. Multilingual code clone detectors make it easier to add new language support by providing syntax information of the target language only. To address the shortcomings of existing multilingual detectors for language scalability and detection performance, we propose a multilingual code block extraction method based on ANTLR parser generation, and implement a multilingual code clone detector (MSCCD), which supports the most significant number of languages currently available and has the ability to detect Type-3 code clones. We follow the methodology of previous studies to evaluate the detection performance of the Java language. Compared to ten state-of-the-art detectors, MSCCD performs at an average level while it also supports a significantly larger number of languages. Furthermore, we propose the first multilingual syntactic code clone evaluation benchmark based on the CodeNet database. Our results reveal that even when applying the same detection approach, performance can vary markedly depending on the language of the source code under investigation. Overall, MSCCD is the most balanced one among the evaluated tools when considering detection performance and language extensibility.

1. Introduction

Code clones are pairs of code snippets that are identical or similar. They are mainly introduced by copy-and-paste programmers while using code generators and so on can also create them (Roy and Cordy, 2007). Although code cloning can increase the speed of software development, code clones are considered bad smells that reduce code quality (Fowler, 1999). They may lead to the propagation of defects and difficulty in software maintenance because any modification would have to be adopted to their clones (Roy and Cordy, 2007). Therefore, code clones require to be detected automatically and managed.

Since the 1990s, various code clone detectors that have made good progress in scalability and accuracy (recall and precision) have been proposed. State-of-the-art tools are known to achieve clone detection for repositories with 100MLOC and provide high-accuracy near-miss clone detection (Li et al., 2020; Alomari and Stephan, 2022; Wang et al., 2023; Sajnani et al., 2016).

Similarly, language extensibility is also a critical evaluation index for clone detection approaches (Roy and Cordy, 2007). The language extensibility of a code clone detector refers to its ability to support multiple languages and extend to new languages. The proposition of new programming languages and the release of existing languages are frequent. According to our previous survey (Zhu et al., 2022), most popular languages maintain major releases (more likely to include syntax updates) at least once every two years, including languages that have been available for less than eight years. However, it is impractical for existing code clone detectors to support multiple languages because they have to adapt their source-code handler program, which is a significant burden to researchers or users requiring specialized knowledge or time. Hence, most code clone detectors only support a limited range of programming languages and often lack support for the latest language versions. Among the 13 clone detectors proposed from 2013 to 2018, 12 mainly supported C or Java (Ain et al., 2019).

To address the issue of insufficient language extensibility, several multilingual code clone detectors have been developed (Roy and Cordy, 2008; Semura et al., 2018). These detectors allow new language support options by simply modifying the configuration files without altering the

☆ Editor: Dr. Martin Pinzger.

* Corresponding author.
E-mail address: zhuwqing1995@ertl.jp (W. Zhu).

https://doi.org/10.1016/j.jss.2024.112215
Received 30 July 2023; Received in revised form 31 July 2024; Accepted 9 September 2024
Available online 5 October 2024
0164-1212/© 2024 The Author(s). Published by Elsevier Inc. This is an open access article under the CC BY-NC-ND license (http://creativecommons.org/licenses/by-nc-nd/4.0/).

tool's code by reusing existing grammar definition files. NiCad (Roy and Cordy, 2008), a well-known clone detector, allows users to specify the analysis method for each language based on TXL (Cordy, 2006). However, writing TXL syntax for a new language remains challenging for users without expert-level knowledge. Additionally, creating new TXL files for code clone detection is unrealistic for maintenance developers because of time constraints. Therefore, the language scalability of NiCad under application is lower than that in theory. A subsequent of the CCFinder series, known as CCFinderSW (Semura et al., 2018), has achieved a lexical analysis mechanism by providing the grammar of comments, identifier names, and keywords of the targeted language. However, CCFinderSW does not support some languages, such as Lua, and cannot detect a majority of Type-3 clones (details in Section 6).

This study aims to develop and evaluate a multilingual code clone detection technique. We address the shortcomings of NiCad and CCFinderSW and implement a multilingual syntactic code clone detector (MSCCD), which can detect Type-3 clones for the most popular languages. We answer the following research questions using MSCCD and other code clone detectors.

- **RQ1**: How does MSCCD perform in recall, precision, and scalability compared with the state-of-the-art tools?
- **RQ2**: How do the performances of code clone detection approaches differ for various languages?
- **RQ3**: How extensible are the existing source-code handlers of the multilingual code clone detectors?

The language processing mechanism of MSCCD is implemented based on ANTLR, a widely-used LL parser generator (Parr and Quong, 1995). To support a new language, MSCCD only requires modification of the configuration file to import the ANTLR grammar definition for the target language. The definitions for most common languages are available in Grammars-v4.[1] The MSCCD uses the ANTLR-generated parser for each source file to obtain a parse tree (PT), from which code blocks are extracted as clone candidates. This study presents a simplified parse tree (SPT) algorithm designed explicitly for MSCCD and a keyword-based filter to remove invalid results from the code-block extraction process. Subsequently, MSCCD converts the code blocks into token bags and detects clones by computing overlap similarity between the token bags. This approach has shown excellent accuracy and scalability in a previous study (Sajnani et al., 2016).

Due to lacking a multilingual clone detection evaluation benchmark, most existing studies have mainly focused on Java for evaluation (Sajnani et al., 2016; Wang et al., 2018; Alomari and Stephan, 2022; Nakagawa et al., 2021; Li et al., 2020). However, there is no general method for predicting if a technique's performance would be consistent across different languages. As a solution, we propose two benchmarks to evaluate the precision and recall of clone detectors for four languages: Java, Python, C, and C++. We used the CodeNet database (Puri et al., 2021), which mainly contains data from two online judge systems (OJ system) for competitive programming: AIZU OJ[2] and AtCoder[3]. Among the data, code pairs accepted by the same OJ problem were regarded as code clones (at least Type-4 clones). Based on this, we generated a benchmark by selecting 12 sub-datasets that could reflect those parts of the edit distance similarity covered by the target clone detector. Thus, we generated a benchmark to check the correctness of the target clone detector by randomly selecting OJ problem pairs. When OJ problems differ, if parts of the problems are similar, code clone detectors at the level of code snippets or functions might consider these parts as code clones. We selected 90 OJ problem pairs and categorized them into three groups based on their Jaccard text similarity of problem description text: 0%–30%, 30%–60%, 60%–90%, and use these as datasets to evaluate the precision of each tool. In addition to MSCCD, we evaluated CCFinderSW, NiCad, and SourcererCC by these benchmarks. The benchmark can be accessed at GitHub.[4]

For this study, we followed the BigCloneBench-based method and evaluated the recall and precision of the MSCCD for comparison with numerous existing clone detectors (Sajnani et al., 2016). We also compared MSCCD with the other multilingual code clone detectors, NiCad and CCFinderSW, regarding language scalability and detection accuracy. We showed that MSCCD was the most balanced detector because it supported the greatest number of languages and had high detection accuracy. However, MSCCD is still not the perfect choice. First, the performance of the ANTLR-generated parser has to be improved. It is prone to many parsing errors, affecting the detection results. Second, MSCCD does not have a near-perfect recall in ST3 (defined in Section 2) and later categories and cannot detect large-variance clones. In the writers' view, an intermediate layer is required for clone detection in the future, as it advances the utilization of the official parser by specifying the standard output format. Furthermore, the intermediate layer allows researchers to apply their code clone detection techniques to more programming languages easily. The language portability or discrimination problem can be recognized and dealt with for code clone detection techniques, including language-dependent parts.

The main contributions of this study are listed as follows.

- We implemented a tool known as MSCCD,[5] (Zhu et al., 2022) which can detect Type-3 clones for a target language by providing its ANTLR grammar-definition file.
- Based on the evaluations, MSCCD supported most of the widely used languages and had a competitive performance with the state-of-the-art Type-3 clone detectors.
- We constructed two clone-detector evaluation benchmarks for Java, C, C++, and Python using the CodeNet dataset. One evaluated the recall and classified the results from Type-1&2 and Type-3&4. The other one evaluated the precision of the clone detectors.
- A comparative study using the proposed multilingual benchmarks showed that recall and precision of clone detection implementations can also vary with language.
- MSCCD was the most balanced detector compared to the other multilingual clone detectors. In the future, additional studies are required to advance the use of official parsers and more multilingualization of detection practices.

The rest of this paper is organized as follows. Section 2 introduces the background and our motivations. Section 3 discusses the implementation of MSCCD. Section 4 introduces evaluations based on BigCloneBench. Section 5 describes the implementation of our multilingual code clone detector evaluation benchmark and the results of evaluating four clone detectors. Section 6 presents a comparative study of three multilingual code clone detectors in their language extensibilities. Section 7 discusses the better multilingual code clone detection approach, the real-world applicability of MSCCD, and threats to validity. Section 8 introduces related works, while Section 9 presents our conclusions and future plans.

2. Background

2.1. Terminology

In this paper, we have used the following terminologies and considered their respective definitions.

Token bag: A multi-set of keywords, identifiers, and literals.

[1] https://github.com/antlr/grammars-v4.
[2] https://judge.u-aizu.ac.jp/.
[3] https://atcoder.jp/.
[4] https://github.com/zhuwq585/MCCD_Benckmarking.
[5] https://zhuwq585.github.io/MSCCD/.

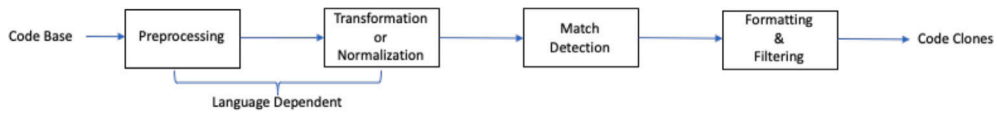

Fig. 1. General clone detection process.

Code segment: A section of contiguous lines of code defined by the quaternion (f, s, e, g), with the source file f, start line index s, stop line index e, and granularity value g.

Code block: A code segment whose sentences are grouped by one grammar rule.

Composition: The composition of a language is defined as a code block corresponding to a party of grammar rules. Classes, condition statements, loop statements, or functions can form a composition. It is mainly used to evaluate the ability of MSCCD to generate token bags for a programming language, as discussed in Section 6.

Types of code clones (Roy and Cordy, 2007; Svajlenko and Roy, 2015):

- **T1 (Type-1)**: These have identical code segments, except for the differences in white space, layout, and comments.
- **T2 (Type-2)**: These have identical code segments, except for the differences in identifier names and literal values, along with the T1 clone differences.
- **T3 (Type-3)**: These have syntactically similar code segments that differ at the statement level. The segments that have statements added, modified, and/or removed with respect to each other and the T1 and T2 clone differences. T3 clones are classified by their syntactic similarity θ:
  - **VST3 (Very strongly Type-3)**: $\theta \in [90, 100)$
  - **ST3 (Strongly Type-3)**: $\theta \in [70, 90)$
  - **MT3 (Moderately Type-3)**: $\theta \in [50, 70)$
  - **WT3 (Weakly Type-3)**: $\theta \in [0, 50)$
- **T4 (Type-4)**: These have syntactically dissimilar code segments that implement the same functionality.

2.2. Multilingual code clone detection

Programming languages may not be able to satisfy all requirements, including writability, extensibility, and cost (Urma, 2017; Stefik and Hanenberg, 2014). Therefore, numerous languages have been developed according to different purposes. Besides, the existing languages are updated frequently to keep pace with improvements. Our previous survey showed that for the ten most popular languages in September 2021 (ranked by PYPL[6]), nearly all languages had major releases at a frequency higher than once every two years (Zhu et al., 2022), providing a high diversity of programming languages.

Code clone detectors have to deal with a variety of situations. As shown in Fig. 1, pre-processing, transformation, and normalization of the detection process are language-dependent (Roy and Cordy, 2007). When adding a new programming language support to a code clone detector, most operations in these processes require a specific understanding of the language's syntax, such as rules for removing comments, pretty printing source code, tokenization, and parsing. Therefore, an appropriate source code handler has to be developed for the new language to achieve pre-processing, transformation, and normalization. Correspondingly, when the release of an existing language includes syntax modifications, the developer must also modify the source code handler to support the new syntaxes. Therefore, most code clone detectors cannot easily and quickly add support for a new language or new version of an existing language.

[6] https://pypl.github.io/.

Another important reason is the trade-off between language extensibility and the detection accuracy of clone detectors. Current studies are focused on improving the detection accuracy (recall and precision) of T3 and T4 clones (Alomari and Stephan, 2022; Wu et al., 2020; Wang et al., 2023). Other code clone terms, such as gapped clones (Wang et al., 2018; Nakagawa et al., 2021) and near-miss clones (Saini et al., 2018), are also garnering attention. Detecting such more difficult-to-detect parts, which require more and higher-dimensional data and more complex pre-processing processes, is particularly challenging in multilingual contexts, as demonstrated by code clone detection approaches based on program dependence graph (PDG) or program slicing (Alomari and Stephan, 2022; Wang et al., 2017). Naturally, researchers are more concerned about the accuracy of their tools than with the number of languages supported by the tool.

Owing to the reasons mentioned above, the language extensibility of existing tools requires improvement. In the 13 code clone detectors proposed from 2013 to 2018, 12 tools supported a limited range of programming languages, including Java and C/C++ (Ain et al., 2019). In addition, the existing evaluation benchmarks for code clone detection using C and Java as the target languages and lack support for other languages (Svajlenko and Roy, 2015, 2021). These two factors form a vicious circle that hinders research on clone detection approaches considering linguistic diversity. Researchers are highly willing to implement their clone detection approach with benchmark-supported languages in this context. Therefore, benchmarks for other languages are unnecessary because most detectors only support a tiny range of benchmark-supported languages.

The lack of support for other languages has caused the research on code clones targeting other languages to stay caught up in progress. Researchers are yet to confirm whether the published detection methods perform as well for other languages. In addition, the focus of detection varies depending on the detection method. (e.g., some methods are good at detecting gapped clones, while others are good at near-miss clones.) With limited choice, or even only their implementation, studies on other languages often apply only one detection method, which may affect their effectiveness (Cheung et al., 2016; Pizzolotto et al., 2022).

Given these challenges, several multilingual clone detectors have been developed. A multilingual clone detector is one that provides a multilingual code-handling mechanism that can change language support without modifying the original program, which achieves high language extensibility. While tools like NiCad (Roy and Cordy, 2008), CCFinderSW (Semura et al., 2018), and some text-based language-independent approaches (Ducasse et al., 1999) have met these definitions, they each have unique strengths and weaknesses. Issues such as T3 clone detection, low precision, and language extensibility (as discussed in Section 7) vary between tools. Thus, achieving the best balance between detection capability and language extensibility proves challenging. In this study, we implement a relatively balanced multilingual code clone detector and evaluate it to derive the direction of the best multilingual code clone detector.

3. Proposed tool: MSCCD

This part aims to develop a tool that detects Type-3 clones and can be easily expanded for various programming languages. We chose the most balanced approach in language extensibility and detection ability. Moreover, we implemented a detector named MSCCD (Multilingual Syntactic Code Clone Detector):

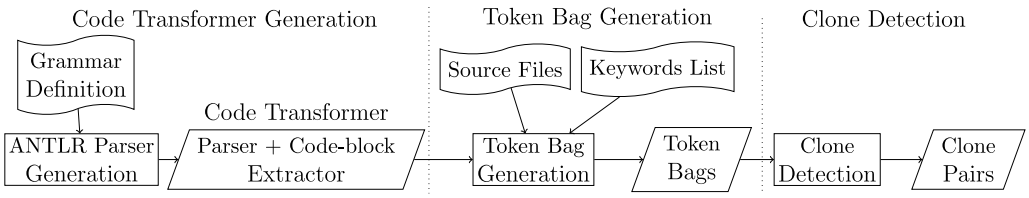

Fig. 2. Overview of MSCCD.

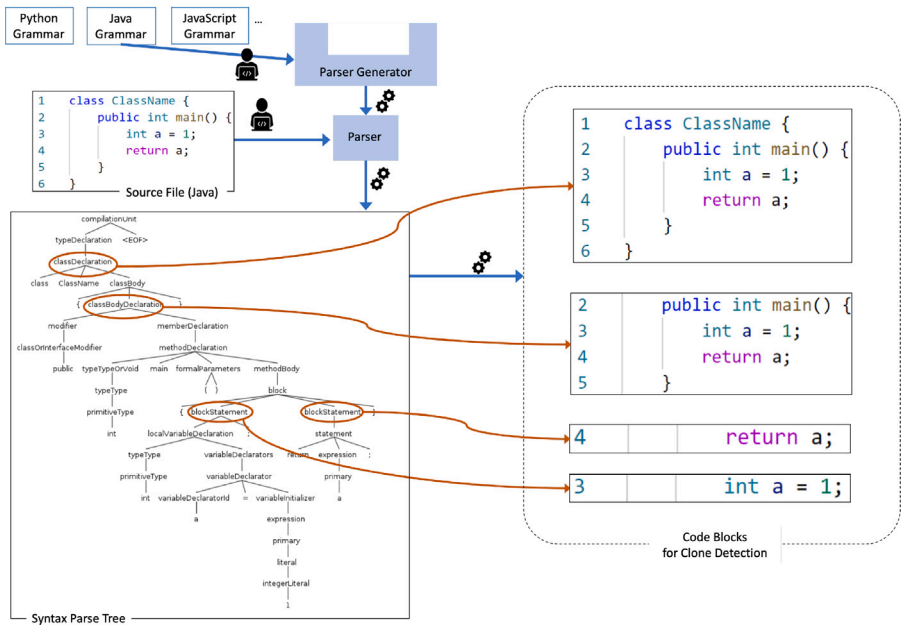

Fig. 3. Multilingual code block partition by parse tree (Zhu et al., 2022).

3.1. Overview of MSCCD

Fig. 2 presents an overview of MSCCD. The system operates three phases: code transformer generation, token bag generation, and clone detection. Initially, MSCCD generates a source code transformer for the targeted language using the ANTLR parser generator. In the token bag generation phase, the code transformer extracts code blocks and converts the code blocks into token bags. Then in the clone detection phase, code clones are identified as similar token bags.

The source code transformer generated in the first phase includes a parser for the targeted language and a code block extractor. ANTLR parser generator allows us to add support for new languages by just providing the grammar definition file of the language without modifying the code of MSCCD. Similarly, providing the new version of the syntax description file is sufficient for syntax updates of the existing languages. The source code transformation approach used in this study relies on a "generic tool-generated syntax tree". It can be realized through the utilization of any parser generator. We chose ANTLR4,[7] an LL-based parser generator, for MSCCD's implementation because of its expansive library of over 150 grammar definition files available in the "grammars-v4"[8] repository. Users can directly leverage these grammar definition files without creating new ones. Furthermore, ANTLR4's rich API helps mitigate implementation challenges.

3.2. Code block extraction using parse tree

The token bag-based clone detection method has been proven to have excellent detection performance by previous studies (Zhu et al., 2022; Nishi and Damevski, 2018). The method can effectively detect Type-3 code clones with high precision and achieve high scalability for repositories with 100MLOC. Other candidate methods perform worse or cannot be applied to general parse trees (PTs). (Note that in this context, we only generate the bare parse tree and do not perform any language-dependent operations based on the semantics of the target language.) In the last two phases, MSCCD converts each source code file into token bags and then detects code clones by similarity comparison. The details are described in the following two subsections.

Extracting code blocks from source code needs careful consideration in multilingual code clone detection. Existing code clone detectors often identify blocks with language-dependent semantics. However, this method could be better in our case as it requires substantial adjustments for each language. To circumvent this issue, we propose representing a semantic code block as each subtree in a PT. Fig. 3 illustrates the process of code block partition. A PT is an ordered tree representing the process of grammar derivation. Each non-leaf node in the PT corresponds to a grammar rule, while each leaf node corresponds to a terminator (lexical units) and a word in the source code. Each subtree whose vertex is a non-leaf node in the PT represents a contiguous part of the source code and corresponds to a rule in the grammar definition. These contiguous parts of code can be functions, code blocks, and statements that are the object of clone detection comparisons. Code blocks can be extracted from the language-independent PT by referring to these subtrees.

However, extracting code blocks corresponding to every subtree for clone detection is insufficient. Plural subtrees may correspond to identical code blocks in PT, shown as Nodes 0 and 1 in the PT of Fig. 4. In the syntax tree of Fig. 3, there is a real example: the node *typeDeclaration* near the root and its child node *classDeclaration* correspond to exactly

[7] https://www.antlr.org.
[8] https://github.com/antlr/grammars-v4.

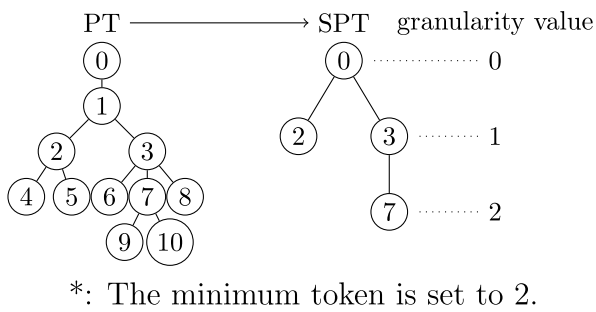

Fig. 4. Simplification of a parse tree.

identical leaf nodes. Grammar rules such as "*statements* : *statement*+" are common. If the non-terminal *statement* on the right side matches only once in the derivation, the final PT will correspond to the same terminators (the same code block) for subtrees with *statement* and *statements* as the vertex. Besides, if a part of the subtrees corresponds to smaller code blocks or even a part of a statement (e.g., Node 9 and 10 in Fig. 4), this part should also be excluded from the comparison of clone detection.

Algorithm 1: Parse Tree Simplification

Input: T is a PT, each tree node contains an attribute *size* representing its token number and an attribute *child* containing child nodes; *mSize* is the configured minimum size
Output: An SPT

```
1  Function ParseTreeSimplification(T, mSize):
2      foreach tree node n in pre-order traversal of T do
3          while n.length == n.child[0].length do
4              merge (n, n.child[0])
5          end while
6          if n.size < mSize then
7              foreach child node cn from n do
8                  delete (cn)
9              end foreach
10         end if
11     end foreach
12     return T
13 end
```

To eliminate these invalid code blocks, we propose algorithm 1 to simplify PT and extract code blocks from them. We traverse each node n the PT in pre-order. If the number of corresponding leaf nodes of n equals n's first child node, this means that the node n only contains one child node $n.child[0]$, and they correspond to the same leaf nodes. Then, n and $n.child[0]$ are merged into one node (lines 3–5). Ideally, $mSize$ represents the minimum number of tokens for a single statement in the program such that all blocks containing more than one complete and consecutive statement are obtained. In practical usage, $mSize$ represents the minimum token for clone detection, where all code blocks containing tokens less than $mSize$ will not be extracted. Therefore, if leaf nodes corresponding to n are less than $mSize$, all the child nodes of n are deleted and will not be traversed (lines 6–10). We call the tree left behind a Simplified Parse Tree (SPT). The subtrees of all nodes in the SPT corresponding to the source PT are the target of code block extraction.

In Fig. 4, we illustrate an example of SPT and introduce our definition of granularity value: the depth of each SPT node. Here, a larger granularity value represents a finer granularity. Furthermore, nodes having the same granularity value are more likely to represent the precise composition of the language (Zhu et al., 2022). For example, in Java, nodes having a depth value of one represent classes.

Additionally, for most languages, certain keywords are essential in defining semantic code blocks, such as "class" for classes, "def" or "function" for functions, and "for" or "while" for loops. These elements are also clone detection targets. Nodes in the PT corresponding to these blocks contain a child node that only contains keywords (Zhu et al., 2022). Taking this into account, we offer a keyword filter to reduce the number of candidates for detecting clones between specific structures like functions.

In the implementation, the input project P consists of a number of source files f, that is, F: $P = \{f_0, f_1, \ldots, f_n\}$. MSCCD partitions each source file f into several token bags Bf_{GV} in each granularity (from 0 to GVmax), $f = \{Bf_0, Bf_1, \ldots, Bf_{GVmax}0\}$, where $Bf_{GV} = \{Bf_{GV}0, Bf_{GV}1, \ldots\}$. We also consider overlaps between token bags corresponding to nodes of different granularities as viable candidates for detection. Algorithm 2 shows the operation on each file f. For each source file f, SPT generated by Algorithm 1 is given as input. For each node, n of the SPT, token bags are generated directly from the subtree corresponding to n. When the keyword filter is activated (a non-empty keywords list is provided), the token types of all the leaf nodes contained in the child nodes of the node n are checked. If any child nodes of n do not contain only keywords, node n is excluded from the code block extraction result.

Algorithm 2: Token Bag Generation

Input: T is a SPT generated by Algorithm 1. K is the keywords list.
Output: A Collection of token bags

```
Function TOKENBAGGENERATION(T, K):
    TargetNodes,TokenBags = [];
    if K == null then
        foreach TreeNode n in T do
            TargetNodes.append(n);
        end foreach
    else
        foreach TreeNode n in T do
            if KeywordsFilter(n, K) == True then
                TargetNodes.append(n);
            end if
        end foreach
    end if
    foreach TreeNode tn in TargetNodes do
        TokenBags.append(token bag created from tn);
    end foreach
    return TokenBags
end
Function KeywordsFilter(n, K):
    foreach TreeNode cn in n.childs do
        flag = 0;
        foreach LeafNode t corresponds to cn do
            if t ∈ K then
                flag |= 1;
            else
                flag |= 3;
            end if
        end foreach
        if flag == 1 then
            return True;
        end if
    end foreach
    return False;
end
```

Table 1
Types and configurations for target detectors.

Tool	Code representation	Configuration
MSCCD	Token bag	min tokens: 20, similarity threshold: 70%, granularity value: 0–4
SourcererCC	Token bag	min tokens: 50, similarity threshold: 70%
CCFinderX	Token sequence	min tokens: 50, min token types: 12
NIL	Token sequence	N value: 5, verification threshold: 0.7 filtration threshold: 0.1
CCAligner	Token sequence	min lines: 6, similarity threshold: 60%, window size: 6, edit distance: 1
SAGA	Suffix tree of token sequence	similarity threshold: 70%, min tokens: 40, function
iClones	Suffix tree of token sequence	min tokens: 50, minblock: 20 tokens
NiCad	Normalized code text	min lines: 6, similarity threshold: 70% blind identifier normalization, identifier abstraction
srcClone	Program slice	similarity threshold: 75%
Deckard	AST	min tokens: 50, similarity threshold: 85%, 2 token stride

3.3. Clone detection

During clone detection, MSCCD calculates the similarity between all candidate token bag pairs. To calculate the similarity, we adopted the overlap similarity method, which has reported a good recall and scalability by SourcererCC (Sajnani et al., 2016). For a given pair of token bags, overlap similarity is defined as the number of tokens they share divided by the total number of tokens in the larger bag. As presented in Eq. (1), if the similarity between token bags B_x and B_y is larger than the threshold θ, the bags are judged as a code clone.

$$OverlapSimilarity\left(B_x, B_y\right) = \frac{\left|B_x \cap B_y\right|}{MAX\left(\left|B_x\right|, \left|B_y\right|\right)} \geq \theta. \quad (1)$$

A challenge in such detection lies in identifying suitable candidates for comparison. In general, clone detectors have to compare every possible code segment against every other potential segment (Roy and Cordy, 2007). However, as introduced in Section 3.2, there may be an overlap between token bags of different granularities. Scoping with all token bags can produce multiple meaningless results, such as identifying a code block as a clone with its subset. The number of token bags generated at multiple granularities is already significantly higher than those generated at a single granularity. Moreover, filtering the meaningless results after detection would add much overhead. These can lead to scalability problems.

Thus, we choose to group token bags by granularity values and detect code clones within each group, as similar components are more likely to appear at the same granularity values. In the case of copy-and-paste, there is a high probability that the newly created clones will have the same granularity values as the source code. When setting the biggest granularity value as g_{max} and the number of token bags in granularity value i as N_i, the time complexity of candidate comparison is reduced from $O\left(\left(\sum_{i=0}^{g_{max}} N_i\right)^2\right)$ to $O\left(\sum_{i=0}^{g_{max}} N_i^2\right)$. Besides, the detection can be easily parallelized.

Consider the case of code clones at different granularity levels. Suppose there is a code block $B1$ with granularity value x, and another code block $B2$ at granularity value $x+1$. In this case, it is highly probable that $B2$'s parent block $B2'$ (of granularity level x) is a clone of $B1$, unless the similarity threshold is set to 100%. Therefore, avoiding comparisons between code blocks of differing granularity values does not significantly decrease recall.

When token bags $B1$ and $B2$, at the same granularity level x, are identified as a code clone, it is highly likely that their sub-bags at the next granularity level $(x+1)$, namely $B1'$ and $B2'$, will also be code clones. For example, corresponding parts of a T1 clone must be a T1 clone. In such a case, MSCCD only reports clones with the smallest granularity value (the coarser granularity).

4. Evaluation using BigCloneBench

As mentioned in Section 2, in recent years, most syntactic clone detectors have been evaluated based on BigCloneBench. To compare with these existing tools, we evaluated MSCCD using the same method with the SourcererCC research based on BigCloneBench (IJaDataset) (Svajlenko and Roy, 2015; Group et al., 2006; Sajnani et al., 2016), which was also used in many studies. We cited these studies which relate to syntactic code clone detections to compare MSCCD with them (Sajnani et al., 2016; Wang et al., 2018; Alomari and Stephan, 2022; Li et al., 2020; Nakagawa et al., 2021). Especially, SourcererCC is the best baseline detector when evaluating MSCCD. Because SourcererCC is well-evaluated and MSCCD uses the same similarity measure as SourcererCC's to verify code clones (see Table 1).

4.1. Recall

We measured the recall of MSCCD using BigCloneEval (Svajlenko and Roy, 2016), which implements a framework of BigCloneBench (Svajlenko and Roy, 2015). BigCloneBench is a benchmark built by real clones collected from IJaDataset (Group et al., 2006), a large inter-project repository consisting of more than 25,000 open-source Java projects. BigCloneBench can report the recall of a code clone detector from Type-1 to Type-4. Specially, Type-3 clones are divided into VST3, ST3, MT3, and WT3 by their syntactical similarity. The evaluated tools targeted syntactic code clones; hence, WT3 and T4 results were excluded from this experiment.

Table 2 lists the results of recall measurements. For all clones, MSCCD had a relatively good recall in T1 (100%), T2 (98%), and VST3 (93%) clones. The recall dropped for ST3 clones (from 93% for VST3 to 62%). For MT3, WT3, and T4 clones, MSCCD faced detection challenges. Similar to most tools, MSCCD performed better in detecting intra-project clones. In this case, MSCCD had a perfect recall for T1, T2, and VST3 clones (100%). The recall started dropping for the MT3 clones (from 88% for ST3 to 26%). For inter-project clones, the recall of MSCCD was slightly lower than that in the inter-project clones category, while it fell below 50% for ST3 clones (48%).

The recall of MSCCD was similar to that of SourcererCC because MSCCD used the same similarity with SourcererCC (Sajnani et al., 2016). However, the recall of MSCCD for ST3 and MT3 was slightly higher than that of SourcererCC because MSCCD detected code clones in various granularities. For example, in function-level comparisons, a code clone between functions was flagged as a none-clone by SourcererCC and MSCCD. However, MSCCD further compared the similarity between the two functional bodies and detected it as a clone at that granularity. When the line coverage between the two functional bodies was higher than 70% for a metafunction, BigCloneEval still considered the clone to be successfully detected.

Most tools perform well in detecting T1, T2, and VST3 code clones, with seven of 10 tools having a recall of at least 91%. A significant difference was reflected in detecting ST3 clones. For ST3 clones, specialized tools (srcClone and NiCad) achieved a recall of 95%, while CCFinderX, a tool famous for accurately detecting T2 clones, achieved only approximately 15%. MSCCD had an average level of 62% recall for ST3 clones, which ranked fifth. Only SrcClone was able to efficiently detect MT3 clones, achieving a recall of 77%. The recall for MT3 clones by other tools was no more than 12%.

Table 2
Recall measurements by BigCloneEval.

Tool	All Clones					Intra-Project Clones					Inter-Project Clones					Citations
	T1	T2	VST3	ST3	MT3	T1	T2	VST3	ST3	MT3	T1	T2	VST3	ST3	MT3	
MSCCD (Zhu et al., 2022)	100	98	93	62	6	100	100	100	88	26	100	97	85	48	6	–
SourcererCC (Sajnani et al., 2016)	100	98	93	61	5	100	99	99	86	14	100	97	86	48	5	
CCFinderX (Kamiya, 2021)	100	93	62	15	1	100	89	70	10	4	98	94	53	1	1	
Deckard (Jiang et al., 2007)	60	58	62	31	12	59	60	76	31	12	64	58	46	30	12	Sajnani et al. (2016)
iClones (Göde and Koschke, 2009)	100	82	82	24	0	100	57	84	33	2	100	86	78	20	0	
NiCad (Roy and Cordy, 2008)	100	100	100	95	1	100	100	100	99	6	100	100	100	93	1	
CCAligner (Wang et al., 2018)	100	99	97	70	10	–	–	–	–	–	–	–	–	–	–	Wang et al. (2018)
srcClone (Alomari and Stephan, 2022)	100	100	99	95	77	–	–	–	–	–	–	–	–	–	–	Alomari and Stephan (2022)
SAGA (Li et al., 2020)	100	98	91	45	5	–	–	–	–	–	–	–	–	–	–	Li et al. (2020)
NIL (Nakagawa et al., 2021)	100	97	94	67	11	–	–	–	–	–	–	–	–	–	–	Nakagawa et al. (2021)

Table 3
Precision measured by random sampling test for BigCloneBench.

Tool	Precision %	Precision (10 LOC) %	Citations
MSCCD	92–94	90	Zhu et al. (2022)
SourcererCC	83–99	86	Sajnani et al. (2016) and Nakagawa et al. (2021)
CCFinderX	72	79	Sajnani et al. (2016)
Deckard	28–34	30	Sajnani et al. (2016) and Saini et al. (2018)
iClones	91	93	Sajnani et al. (2016)
NiCad	56–99	80	Sajnani et al. (2016), Saini et al. (2018) and Nakagawa et al. (2021)
CCAligner	34–92	83	Wang et al. (2018) and Nakagawa et al. (2021)
srcClone	90	–	Alomari and Stephan (2022)
SAGA	99	–	Li et al. (2020)
NIL	94	–	Nakagawa et al. (2021)

4.2. Precision

Automatic precision measurement is still an open question because of the need for high-quality benchmarks. A general method randomly selects a sample of detected results and involves manual checking for the correctness of each reported clone (Sajnani et al., 2016). Following this approach, we randomly selected 400 clone pairs from the recall measure experiment discussed in Section 4.1, and equally distributed them among five judgers to check the correctness (Zhu et al., 2022). For the reported pairs that were hard to judge, the pairs were marked as unknown and counted as false positives. Owing to the randomness of result sampling, repetition of the experiment would lead to different results. Therefore, we cited multiple results of the target detectors evaluated by the same experiment with the same detector configuration. The true performance of the target detector was somewhere in between.

The biggest challenge with this experiment was the need for a larger sample size. In the case of MSCCD, 61 917 clones included in BigCloneBench were detected. However, MSCCD reported results that were multiple times the number of clones in different granularities, which were not collected in BigCloneBench. The smaller the sample size, the greater the likelihood of being affected by randomness. Therefore, if the random sampling precision experiment was conducted multiple times, results with higher means and fewer fluctuations over multiple trials represented higher precision. Furthermore, we conducted the same experiment in our original paper (Zhu et al., 2022) again for MSCCD. The results are listed in Table 3. We observed that the result for MSCCD was 93%, approximately the same as the average of multiple results of SourcererCC (91%). Among other tools with multiple results, NiCad and CCAligner were considered less accurate than SourcererCC and MSCCD because of their lower average values, although they also included a high result of more than 90%.

Another factor that could affect the validity of the comparison was the difference in detection range. For example, MSCCD reported clones at multiple granularities such as file, block, and function. False positives were more likely to occur between larger code segments. The probability of smaller clones, such as loops in the sample, was better, which could make the evaluation results higher than the actual value. Therefore, some studies set the object range above 10 LOC (Wang et al., 2018; Sajnani et al., 2016). In this group, MSCCD also ranked second among the seven tools.

4.3. Execution time

To evaluate the scalability of MSCCD, we generated test data from 1KLOC to 100MLOC by randomly selecting files from IJaDataset (Group et al., 2006). We used the "cloc" tool to count the lines of code while excluding blank and comment lines (Danial, 2021). The experiment was executed on a machine with an i7-8665U CPU and 12 GB RAM. The setup was similar to that used in the experiment by Sajnani et al. (2016). We counted the execution time of token bag generation and clone detection for each group. The time consumed for tokenizer generation was not counted because only a single execution was conducted, which only took less than 5 s. The configuration of MSCCD was the same as that discussed in Section 4.1.

Table 4 lists the result. We cited the scalability evaluation results of six clone detectors from two existing studies (Sajnani et al., 2016; Wang et al., 2018). The results for MSCCD and CCAligner are derived from their respective experiments, while the results for the other five tools come from experiments reported in the SourcererCC paper (Sajnani et al., 2016). The machine used for testing MSCCD is comparable to the machines used in these two studies. Table 4 do not include the result of NIL, SAGA, Oreo, because the machines they used were not comparable. It also excludes srcClone because the same experiments were not conducted. MSCCD was the fastest for 100KLOC to 10MLOC. However, MSCCD was slightly slower than the others for 1KLOC to 100KLOC, owing to the overhead of parser execution. Also, due to memory requirements, MSCCD could not complete the detection for 100MLOC. In such an implementation, MSCCD failed to optimize memory usage sufficiently. Considering the situation of SourcererCC, MSCCD has the opportunity to achieve the scalability of 100MLOC through further optimization.

The answer to RQ1: Compared with existing clone detectors, MSCCD has a median level of recall, precision and scalability.

5. Multilingually measuring recall&precision of code clone detectors

MSCCD supports numerous languages, making evaluating its efficacy across this diverse linguistic landscape crucial. To address this

Table 4
Execution time measurement.

(a) Used machine in each detectors' experiment

Detector	Machine	Citations
MSCCD	i7-8665U (quad-core) CPU, 12 GB RAM	–
NiCad SourcererCC CCFinderX Deckard iClones	3.5 GHz quad-core i7 CPU, 12 GB RAM	Sajnani et al. (2016)
CCAligner	quad-core CPU, 12 GB RAM	Wang et al. (2018)

(b) Execution time

Detector	1K	10K	100K	1M	10M	100M
MSCCD	5 s	5 s	7 s	32 s	15 m 9 s	MEMORY
NiCad	1 s	4 s	21 s	4 m 1 s	11 h 42 m 47 s	INTERNAL LIMIT
SourcererCC	3 s	6 s	15 s	1 m 30 s	32 m 11 s	1 d 12 h 54 m
CCFinderX	3 s	4 s	21 s	2 m 18 s	28 m 51 s	3 d 5 h 49 m
Deckard	2 s	9 s	1 m 34 s	1 h 12 m 3 s	MEMORY	–
iClones	1 s	1 s	2 s	MEMORY	–	–
CCAligner	1 s	1 s	7 s	1 m 13 s	24 m 56 s	–

s: seconds, m: minutes, h: hours.

need and investigate potential performance variations in the same detection method across different languages, we have constructed two benchmarks using data from CodeNet,[9] an online judge System. These benchmarks serve as valuable tools to assess the recall and precision in the four languages of the target detector, offering insights that extend beyond the constraints of single-language evaluations.

5.1. Introduction of the benchmark

Constructing an evaluation benchmark for code clone detectors is an arduous task owing to the requirement of a large code base and marking all true clones in the code pairs. No clone detector with 100% recall and precision exists, while manual marking of all the actual clones is nearly impossible. The most significant existing benchmark, BigCloneBench, combines heuristics and manual annotation of many function-level clones (Svajlenko and Roy, 2015). However, it only supports Java. Therefore, producing recall–precision benchmarks from real-world code is a near-impossible task.

With the rise of AI for Code research in recent years, large-scale annotated datasets are needed for training machine learning models and benchmarking evaluations. Datasets that contain data from Online Judge systems (OJ systems) play an important role, e.g., CodeNet and CodeXGLUE (Puri et al., 2021; Lu et al., 2021). An OJ system can compile and execute the submitted code and test it to judge whether it is a successful submission for competitive programming. It can provide information such as source code, task descriptions, execution results, etc. These data can also evaluate the code clone detector's recall and precision. All code pairs accepted by the same task can be considered as at least T4 clones. The more clones the target detector reports, the higher recall it has. On the other hand, submissions between tasks that are not identical are not code clones. Nevertheless, they can also be misreported as clones because they can still contain similar syntacticians (e.g., same methods, variable names, or similar program patterns). The more such cases (false positive), the lower the precision of the target detector.

For this study, we chose CodeNet to implement our ideologies. CodeNet was proposed by IBM. It has a collection of over 13,916,000 submissions in 4053 problems from AIZU Online Judge[10] and AtCoder.[11] CodeNet contains data from over 50 programming languages. While most data originate from C, C++, Java, and Python, it is already the largest database of supported languages. Besides, identical problems have also been well-labeled.

Fig. 5 shows the process of generating benchmarks for evaluating recall and precision using data from CodeNet. We first pre-processed all the source files to remove all comments and contiguous breaks to avoid mismatches of clone matcher (Step 1 in Fig. 5). For evaluating recall, we selected 14 OJ problems from CodeNet and calculated the similarity of all the code pairs accepted by each problem (Step 2). The similarity of code pairs from these 14 OJ problems covered all cases from 20% to 100%. The code submitted to every two problems constituted a subset for evaluating precision. We calculated the similarity for every pair of OJ problems by calculating the Jaccard similarity between their descriptive texts (Step 3). We set up three groups according to the similarity of the problems, and each group randomly selected 30 pairs of problems to generate a total of 90 subsets for each language (Steps 4–5).

Using our created benchmarks to evaluate code clone detectors, we used a clone matcher similar to BigCloneBench: line coverage clone matcher (Svajlenko and Roy, 2015). If the reported clone covered more than a certain percentage of lines of the clones in the benchmark, the clone in the benchmark would be recognized as detected. In this study, we used a clone matcher with 70% coverage. Hence, we could only determine code clones between files. Clone detectors that do not support file-level detection could result in some correct clone reports not being matched by our benchmark. Therefore, we used the 70% configuration, though we believe a higher line coverage threshold should have been used.

We evaluated four detectors: MSCCD, NiCad,[12] CCFinderSW,[13] and SourcererCC.[14] The first three are existing multilingual clone detectors, while SourcererCC also supports C, C++, and Java in the benchmark. We used the language support provided by the authors and did not add any changes. The configurations for each detector are listed in Table 5. We set the minimal size of tokens to 20 to match our benchmark. For the other items, we used the default settings of each detector. Although the research from Ragkhitwetsagul et al. suggests that the default configuration of clone detection tools may not be optimal configuration (Ragkhitwetsagul et al., 2018), we opted to use the default settings of each tool in order to facilitate comparison with the evaluation results from existing studies. Furthermore, in recall measurement in Section 5.2 and precision measurement in Section 5.3,

[9] https://github.com/IBM/Project_CodeNet.
[10] https://onlinejudge.u-aizu.ac.jp/home.
[11] https://atcoder.jp/.
[12] NiCad: https://www.txl.ca/txl-nicaddownload.html.
[13] CCFinderSW: https://github.com/YuichiSemura/CCFinderSW.
[14] SourcererCC: https://github.com/Mondego/SourcererCC.

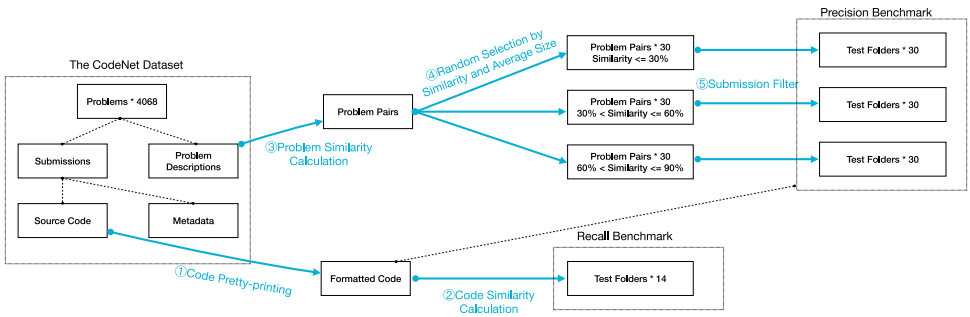

Fig. 5. Process of evaluating recall&precision in various languages.

Table 5
Configurations for the target detectors.

Tool	Configurations
MSCCD	min tokens:20, similarity threshold:70%, granularity:file
SourcererCC	min tokens:20, similarity threshold:70%, granularity:file
NiCad	min lines:6, similarity threshold:70%, blind identifier normalization, identifier abstraction, granularity:file
CCFinderSW	min tokens:20

we set a limitation for the benchmarks with a minimum token count of 20 and a minimum line count of 6.

5.2. Benchmarking of recall

Existing recall evaluation benchmarks are dedicated to grouping clones according to their type and reporting recall separately for each type of clone (Svajlenko and Roy, 2015, 2021). However, existing clone detectors can automatically distinguish between T1 and T2 only. Manual annotation of T3 and T4 clones is almost impossible due to the unrealistic workload.

Hence, we attempted to evaluate the recall of a detector from another perspective. Numerous similarities can be used to evaluate the similarity between two code segments, including edit distance. Edit distance presents the minimum number of operations required to transform one string to another. The fewer the changes made to the source code, the more similar the new code segment is to the source code, which fits very well with the process of code clone generation. In light of these considerations, the proposed benchmark evaluates the coverage of current detection methods for edit distance-based similarity.

$$LevTS(A,B) = \begin{cases} |A| & if|B| = 0 \\ |B| & if|A| = 0 \\ LevTS(tail(A), tail(B)) & A[0] = B[0] \\ 1 + min \begin{cases} LevTS(tail(A), B) \\ LevTS(A, tail(B)) \\ LevTS(tail(A), tail(B)) \end{cases} & otherwise \end{cases} \tag{2a}$$

$$LevTS(A,B) \epsilon [0, max(|A|, |B|)] \tag{2b}$$

$$LevSimi(A,B) = \frac{Max(|A|, |B|) - LevTS(A,B)}{Max(|A|, |B|)}, LevSimi(A,B) \epsilon [0,1] \tag{3}$$

We used the Levenshtein distance, an edit distance that allows three operations: insertion, deletion, or substitution (Navarro, 2001). We applied the distance to token sequences instead of strings. The Levenshtein distance $LevSimi_{(A,B)}$ of two token sequences A and B was calculated using Eq. (2). The edit distance similarity between the two token sequences A and B was defined in Eq. (3), such that one minus the edit distance of A and B was divided by the length of the more extensive token sequence. In addition, most modern code clone detectors can accurately detect T1 and T2 clones. Therefore, we normalized all identifiers as the same token before computation. Thus, T1 and T2 clones were both computed with a 100% similarity.

We calculated the similarity of all code pairs in all successful submissions for the first 2300 problems and counted their distribution from 0.20 to 1.00. Thereafter, we chose 14 problems with the full distribution of LevTs Θ in the collection $\{0.20 \leq \Theta \leq 1.00 \mid \Theta mod 0.01 = 0\}$ for all targeted languages. Fig. 6 shows the distribution of the 14 problems across a total of 56 sub-datasets in the four target languages for the number of code pairs and the average number of tokens. C++ had the highest number of files, while Java had relatively longer programs. Python had significantly fewer average tokens than the other languages.

For each evaluation object (detector-language), we performed clone detection on each of the 14 sub-datasets, and then the final results were aggregated. We reported the recall for each group in groups at 0.20 intervals, which are presented in Table 6. The parts with 100% similarity were T1 and T2 clones, while the other corresponded to T4 and T3 clones. For areas of T1 and T2, CCFinderSW (famous for detecting T1 and T2 clones) ranked first for Java and Python. CCFinderSW also got a near-perfect result of 98% for C and ranked 2nd behind NiCad in this area. MSCCD ranks second in C++, but the gap between it and the top-ranked SourcererCC is extremely small, with less than a 1% difference in the number of detected clones. In the T3 and T4 areas, the recall of all four tools began to show a significant decline. For the LevSimi interval between 80% to 99%, MSCCD and SourcererCC ranked in the top two in the three languages other than Python, while NiCad ranked first in Python. For the intervals under 79% similarity, none of the detectors had a recall of more than 40% for all languages, while MSCCD ranked 1st for the most time in these intervals.

For the same detectors, performance differences between languages were also evident. For example, during the interval [0.8,0.99], MSCCD performed best for Java (81%), followed by C++ (73%) and C (70%), and was relatively weakest for Python (42%). NiCad also had a significant difference in this interval, with a 49% recall for Python and C, while the recall for Java is 15%. SourcererCC showed results that were similar to MSCCD because we believe that as statically strongly typed languages, Java, C, and C++ have a higher rate of keyword occurrences in their syntax, such as the description of classes, functions, and types of variables, that assist the detector in identifying clones. Since Java naturally shares more tokens for similar programs and thus has a higher degree of similarity because it has a fixed class, differences between Java and C/C++ were observed. Contrarily, C/C++ source programs contain more pre-processing commands, which brings more dissimilarity.

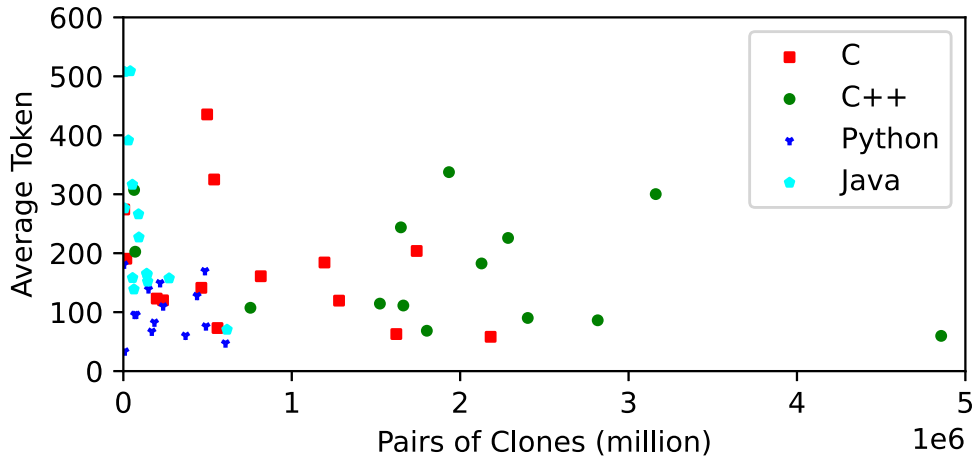

Fig. 6. Distribution of four target languages in pair number and average token.

Table 6
Recall measurement grouped by LevSimi range.

Language	Tool	Recall for each similarity range									
		T3 or T4								T1 or T2	
		[0.20, 0.39]		[0.40, 0.59]		[0.60, 0.79]		[0.80, 0.99]		1.0	
		%	#	%	#	%	#	%	#	%	#
Java	MSCCD	**1**	1940	2	13,353	28	101,345	81	275,952	90	16,417
	SourcererCC	1	1905	**2**	15,581	**29**	102,499	**82**	278,953	90	16,281
	NiCad	0	0	0	16	1	1969	15	52,507	99	18,000
	CCFinderSW	0	0	0	0	0	1188	22	76,052	**100**	18,163
Python	MSCCD	**0**	792	**1**	16,106	**9**	39,906	42	26,046	79	2776
	SourcererCC	–	–	–	–	–	–	–	–	–	–
	NiCad	0	2	0	768	2	10,053	**49**	30,303	98	3447
	CCFinderSW	0	3	0	310	1	4417	21	13,184	**99**	3492
C	MSCCD	**8**	153,559	**10**	419,469	**37**	844,211	70	1,455,265	78	107,235
	SourcererCC	7	140,581	9	392,206	37	837,573	**70**	1,462,909	78	107,361
	NiCad	0	0	0	669	3	77,009	49	1,029,109	**100**	136,913
	CCFinderSW	0	0	0	4	0	9118	17	346,114	98	134,722
C++	MSCCD	**0**	16,756	**3**	345,554	**34**	1,466,095	**73**	1,415,698	90	75,024
	SourcererCC	0	10,897	2	216,535	28	1,208,668	71	1,383,606	**90**	75,609
	NiCad	–	–	–	–	–	–	–	–	–	–
	CCFinderSW	0	0	0	53	0	1188	6	124,825	84	70,275

%: Recall in percentage, #: Number of clones detected.

A noteworthy observation in the results of CCFinderSW was that the C++ trials of CCFinderSW had only 84% recall at the Type-1&2 area. This may have been due to the high frequency of the standard input–output stream in the OJ system. For example, there was no significant difference in the percentage of using C standard statements, such as the printf method from "stdio.h", and C++ standard statements, such as the cout method from "iostream" among C++ programs in CodeNet. This led to several false negatives in CCFinderSW based on token sequence detection of clones.

For a better visual representation of the degree of coverage of edit distance-based similarity by each detection method, we plotted a line graph, as shown in Fig. 7. The horizontal axis represents the edit distance similarity, and the yellow bars represent the number of clones included in the benchmark under that similarity. The higher the number of clones, the more likely the evaluation results at that location would have increased the confidence. Moreover, the dots on the dash represent the recall of the tool at that edit distance similarity. We observed that MSCCD and SourcerCC had the same performance and had a relatively higher reproducibility in Type-3 detection compared to NiCad and CCFinderSW. Large fluctuations in individual similarities (even at highly similar positions) were observed because the similarities used by both CCFinderSW and NiCad were very sensitive to changes at the statement level, especially when the changes were located in the middle of the program. Moreover, such types of code clones were unevenly distributed in the benchmark, thus producing significant fluctuations.

5.3. Benchmarking of precision

For precision evaluations, we constructed a benchmark based on "true positives if two blocks of clones are detected from the same problem, and false positives if they are from different problems". The basic process of the experiment was to randomly select a certain number of problems from more than 4000 problems included in CodeNet and gather all the source files into a folder. Thereafter, each target detector was used to detect clones of each folder. The detected clones were considered true positives if they were from the same problem or false positives in case they were from different problems. The use of meaningless variable names, such as `i` and `j`, and some common methods, such as reading from the standard input or printing to the standard output, were widespread in programs submitted to the OJ system. Thus, code pairs from different problems still had the potential to be mistakenly reported as code clones by the clone detector. Thus, a challenge that affected the correctness of the clone detector was observed. The experiment evaluated how the code clone detectors coped with such a situation.

We used the Jaccard similarity to calculate the similarity between the descriptive texts of the problems. The higher the similarity between

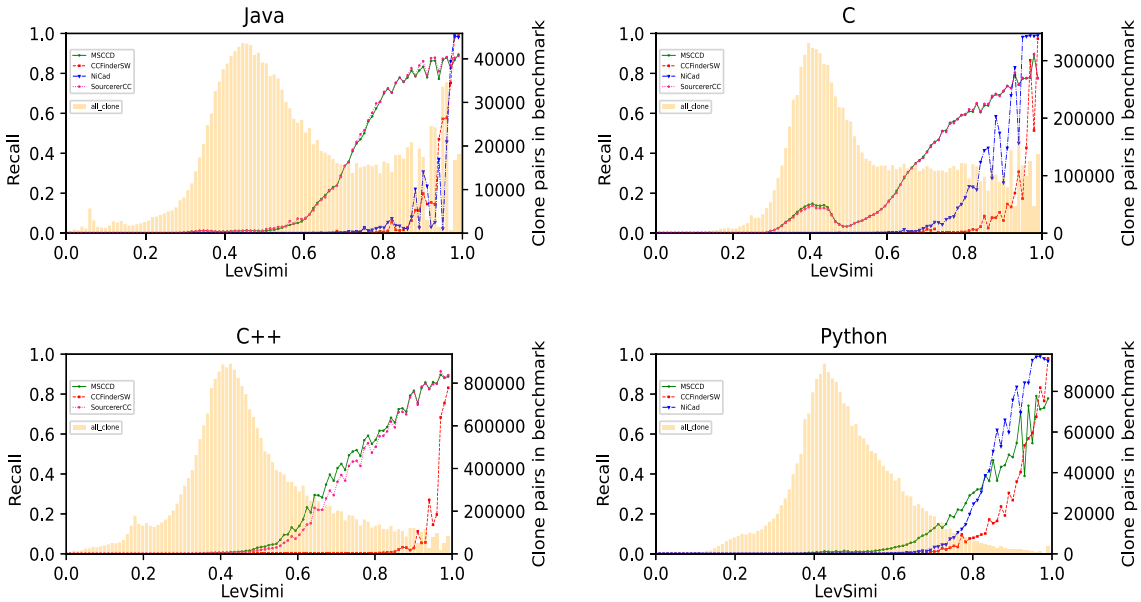

Fig. 7. Recall at each LevSimi.

problems, the more likely it is for the code from these problems to be falsely reported as code clones. In other words, this pair of problems represents a more challenging case for precision testing. Only a part of the problem statement was involved in the computation for each problem. Constraints, input, output, and sample were not computed. We randomly sampled three groups with the similarity σ of $\sigma \in [0, 0.3)$ (group I), $\sigma \in [0.3, 0.6)$ (group II), and $\sigma \in [0.6, 0.9)$ (group III), and each III consisted of 30 pairs of problems. Overlap was not allowed between pairs in the same group. In addition, the difference in the average code length per pair of problems did not exceed 30%.

The results are presented in Table 7. All target tools were performed with near-perfect precision for groups I and II. The similarity of the problems in these two groups was less than 60%. Clone detectors worked well even when the same identifier name or function was applied. The differences between code clone detectors and languages mainly appeared in group III. The problem pairs in group III were more similar to groups I and II, and most were modified versions of existing problems. For example, problems p01950 and p01964 were aimed at implementing a breadth–firth search. P01964 was based on a directed graph, while p01950 was based on an undirected graph. There were many submissions for this class of problems based on modifications of the previous submissions. The changes were usually the addition of a function and a shift in the main function. The class constituted some large-gap clones, although they were considered non-clones by benchmark because they were from a different problem.

The results for MSCCD and SourcererCC were very close, although SourcererCC did not support Python, and both achieved relatively high precisions in Python and C++ (above 85%), followed by Java (slightly below 80%), and the lowest precision for C (slightly below 65%). For the class "large-gap-clone-like clone" mentioned previously, the detection algorithm of MSCCD and SourcererCC were likely to be reported as code clones.

Conversely, CCFinderSW still achieved near-perfect precisions in Java, Python, and C. However, since CCFinderSW cannot detect Type-3 code clones, the parts it reported were relatively less likely to be false positives because of their higher syntactic similarity. Since the detection algorithm of CCFinderSW is susceptible to statement insertion, many false positives were reduced when compared to MSCCD and SourcererCC. The exception was for C++, where the precision of CCFinderSW (70.1%) was significantly lower than that of MSCCD and SourcererCC, which was related to the clone-matcher setting used in the benchmark. In the CodeNet dataset, C++ programs more commonly had longer pre-process commands or macro definitions. Hence, even though there were statement-level differences in the functions, many #include, and #define statements were matched, allowing many results to be matched by the clone matcher and counted as false positives.

Table 7
Precision measurements grouped by similarity of problem description.

Language	Tool	Precision (median)/%		
		Group I (0,0.3)	Group II (0.3,0.6)	Group III (0.6,0.9)
Java	MSCCD	99.7(100)	96.8(100)	78.0(67.7)
	SourcererCC	98.5(100)	97.0(100)	78.9(64)
	CCFinderSW	99.8(100)	94.5(100)	99.9(80)
	NiCad	100(100)	100(100)	95.1(80.8)
Python	MSCCD	99.9(100)	99.8(100)	86.7(83.3)
	SourcererCC	–	–	–
	CCFinderSW	99.9(100)	99.9(100)	99.2(100)
	NiCad	100(100)	100(100)	99.9(100)
C	MSCCD	100(100)	99.6(100)	64.1(57.0)
	SourcererCC	99.7(100)	99.7(100)	64.3(57.1)
	CCFinderSW	100(100)	99.9(100)	99.9(99.9)
	NiCad	100(100)	99.9(100)	70.1(100)
C++	MSCCD	99.8(100)	99.8(99.9)	86.6(68.1)
	SourcererCC	99.4(100)	91.0(99.9)	86.5(60.7)
	CCFinderSW	99.9(100)	99.5(100)	70.1(66.7)
	NiCad	–	–	–

NiCad had results similar to CCFinderSW for Java and Python, but the precision was decreased for C (70%). MSCCD and SourcererCC also had the lowest precision in C, which may have been due to the more significant proportion of indistinguishable clones among the 30 pairs of problems in the C language of the group III. The similarity of the problem description texts and the average size of the submissions were considered in extracting the problem pairs. Hence, the selection of the same problem pairs for all four languages was impossible. To avoid this effect, we also recorded the median results in each group. In contrast to the general case, the median precision for the C language of NiCad was higher than its ensemble value. Thus, the presence of pairs with

Table 8
Language extensibility of the existing tools.

Tool	Conditions for supporting the language	Positive rate of evaluation experiment
MSCCD	Available ANTLR grammar definition files	89.58% (43/48) (18 languages file-level only)
CCFinderSW	Grammar of comment and string can be converted into regular expressions	84.62% (44/52)
NiCad	Available TXL grammar definition files	–
Tool	**Number of languages currently supported**	**Available resources**
MSCCD	25	More than 100
CCFinderSW	15	More than 100
NiCad	10	22

more submissions with a higher percentage of indistinguishable clones was observed, which affected the ensemble results.

> The answer to RQ2: All the evaluated detectors showed different performances between languages in recall and precision, and the difference in performance between languages in recall was more pronounced than precision. Users should try different settings for different languages (mainly including similarity thresholds) to achieve more accurate detection results.

6. Language extensibility of existing multilingual code clone detectors

This section discusses the language extensibility of the existing multilingual clone detectors. Any code clone detection approach can be applied to any language with sufficient effort and time. Owing to the development costs and required knowledge, most code clone detectors have not been extended to support any new languages since their first release. Therefore, we define the language extensibility of a multilingual clone detector for a number of languages that can be supported without having to write or modify the program. We count the number of languages supported by these tools. Moreover, we discuss the language extensibility of MSCCD, CCFinderSW, and NiCad, considering the conditions for supporting a new language, the difficulty, the languages supported by the current version, and the available grammar definition resources. The results are listed in Table 8.

6.1. MSCCD

MSCCD showed conditional support for the targeted language based on the correctness of the ANTLR4 grammar definition file. If an incorrect grammar definition file is used (e.g., due to a version mismatch or inclusion error), it could result in the failure of the parser generation, trigger an error in parsing, and lead to an unsuccessful creation of an accurate and complete parse tree (PT). MSCCD could only complete file-level clone detection when the most severe failure occurred. In addition, MSCCD contains the keywords filter, which can be used to speed up the detection by reducing extracted token bags. When the grammar of object composition of the target language did not have the features utilized by the keywords filter, that composition would not be extracted when the filter was activated.

6.1.1. Language extensibility measurement by token bag generation

Similar to previous experiments in our initial paper (Zhu et al., 2022), we randomly selected programs from Rosetta Code[15] to check whether MSCCD could generate the correct token bag for all the target compositions of the targeted language. Rosetta Code is a programming chrestomathy site that collects instances of multiple languages for over a thousand programming tasks. The site primarily shows case programs that are implemented practically and simply without relying heavily on development frameworks. This ensures that the extracted programs contain the target compositions. The target languages for the experiment were those supported by RosettaCode and Grammars-v4 for 48 languages. We first attempted to generate the MSCCD token bag generator, which includes a specific parser for each language. We randomly selected five programs from Rosetta Code for the languages whose token bag extractor was generated successfully. After that, we checked whether they could generate the correct token bag with or without the keywords filter in effect.

Regarding the keywords filter activated, we selected four compositions for each language: class, function, branch, and loop. These items are the most basic and standard parts of languages. For each item, all corresponding grammar was included within the scope of the test targets. For example, all loop items were test objects, including but not limited to the while loop, for loop, and do-while loop. We regarded function items as the most relevant to show the target language's extensibility among the four items. The other three items indicated whether MSCCD could detect code clones at higher granularity. However, MSCCD did not merely support these four items. The other language components could also be extracted depending on the grammar. The item was passed when all corresponding code blocks were extracted correctly, including the corresponding code block, lacking any irrelevant code segments (circle mark; ●). It was regarded as a failure when a target code block failed to be generated correctly without a syntax analysis error (Zhu et al., 2022).

When the keywords filter was unactivated, all compositions were extracted when the PT was generated successfully. Therefore, the results of individual components were not distinguished in this case. We called it a block/statement level. A positive result (circle mark; ●) meant that the ANTLR-generated parser generated the correct PT for all test source files.

Out of the 48 targeted languages, MSCCD did not support five languages: Three failed during parser generation, and two encountered errors during lexical analysis. An additional 18 languages encountered severe syntax analysis errors, and therefore, MSCCD could only perform file-level detection for these. The detailed results for the successfully processed 25 languages are listed in Table 9. When the keywords filter was turned off, all 25 languages were supported. Dart, C++ classes, and objective-C functions were not fully extracted when the filter was used because they used a different pattern in their syntax that the filter could not recognize. The remaining components were extracted successfully.

6.1.2. Details of ANTLR parser errors

In the experiment of Section 6.1.1, some target languages were judged as not supporting the block/statement level due to syntactic analysis errors. In fact, not all errors directly affect MSCCD's support for the language. Different types and locations of errors have varying degrees of impact. If the main part of the parse tree in successfully constructed, syntactic errors may not affect the outcome of token bag extraction.

To evaluate the impact of ANTLR parser errors on token bag generation in practical use, we conducted an experiment targeting open-source software (OSS). We selected five languages from grammars-v4

[15] http://www.rosettacode.org/wiki/Rosetta_Code.

Table 9
Measurements of MSCCD's language extensibility.

Support status	Language number	Notes
Block-level	25	–
File-level only	18	Error in syntactical analysis
Not supported	5	Cursh in parser generation, severer error in lexical analysis

Language	Results of 25 block-level supported languages				
	Keywords filter activated				Keywords filter inactivated
	Class	Function	Condition	Loop	Block/Statement
C	●	●	●	●	●
C#	●	●	●	●	●
C++	⚹	●	●	●	●
Cobol85	–	●	●	●	●
Clojure	–	–	–	–	●
Cmake	–	–	–	–	●
Dart	⚹	●	●	●	●
Erlang	–	–	●	●	●
Go	–	●	●	●	●
Java	●	●	●	●	●
JavaScript	●	●	●	●	●
Kotlin	●	●	●	●	●
Lua	●	●	●	●	●
Objective-C	–	⚹	●	●	●
PHP	●	●	●	●	●
prolog	–	–	–	–	●
Python	●	●	●	●	●
Rexx	–	–	–	–	●
Rust	●	●	●	●	●
Swift	●	●	●	●	●
TypeScript	●	●	●	●	●
VBA	–	●	●	●	●
Verolog	–	–	–	–	●
VHDL	–	–	–	–	●
VB	–	●	●	●	●

●: Positive ⚹: Negative -: Not applicable.

Table 10
Generated token bags in each parser error situation.

(a) Details of targeted open-source software

Language	Version	OSS		
		Name	Files	LOC
Java	Java20	Spring-boot-3.1.3	6888	679,651
JavaScript	ECMAScript 2020	react-18.2.0	1882	450,759
C#	CSharp6	shadowsocks-windows-3.4.3	127	25,099
C++	CPP14	tensorflow-1.4.0	2314	621,573
Python	Python3.6	django-2.0.5	2454	320,912

(b) File numbers and generated token bags

Language	Error type								Error position			
	No Error		*SE* Only[a]		*LE* Only[b]		*LE&SE*		Head[c]		Tail[d]	
	Files	*n*	Files	*n*	Files	*n*	Files	*n*	Files	*n*	Files	*n*
Java	6659	6.38	217	6.33	0	–	0	–	34	6.56	46	4.52
JavaScript	782	9.54	1100	2.21	0	–	0	–	651	1.69	63	6.13
C#	51	8.90	0	–	76	10.01	0	–	76	10.01	0	–
C++	1861	9.46	388	7.73	0	–	65	4.39	77	5.76	91	9.31
Python	2452	8.55	2	8.59	0	–	0	–	0	–	0	–

[a] *SE*: Syntax Error.
[b] *LE*: Lexel Error.
[c] Head: The first error occurs at the first 20% lines.
[d] Tail: The first error occurs at the last 20% lines.
[e] *n*: token bags generated for every 100 lines of code.

that were explicitly marked with language versions and chose an OSS from GitHub based on stars, which match the language version, and containing more than 50 source files. The details of selected OSS are listed in Table 10. The minimum token size of MSCCD was set to 30. We collected parser error messages for each source file and tallied the number of token bags generated under various error statuses.

Table 10 lists the number of files and generated token bags in each parse error situation.

Python exhibited almost no errors. Java and C++ were relatively insensitive to syntactic analysis errors, with no significant decrease in the number of token bags generated when such errors occurred. JavaScript was the most sensitive to syntactic errors: The number of token bags generated per 100 LOC in the presence of syntactic errors was less than a quarter of that generated without errors. The situation with C# was unique, as it produced many lexical errors and no syntactic errors. Upon investigation, these lexical errors were all due to Byte Order Mark (BOM) symbols in the source files, which the lexical

Table 11
Proportion of error messages.

Message type	JavaScript	C++
No viable alternative at input ...	63.46%	8.74%
Mismatched input ...	18.25%	59.11%
Extraneous input ...	15.78%	4.21%
Missing ...	2.27%	27.93%
Others	0.23%	0

analyzer failed to recognize correctly. However, this did not affect the syntactic analysis.

For C++, when lexical and syntactic errors coincided, the average number of token bags decreased from 7.73, when only syntactic errors were present, to 4.39. This indicates that the impact of lexical errors is more significant than syntactic errors. The earlier the occurrence of either syntactic or lexical errors, the greater their impact, a phenomenon that was reflected in the results for both JavaScript and C++.

ANTLR contains an automatic recovery mechanism that allows the parser to continue after finding a syntax error. This mechanism allows the ANTLR parser to scan the entire file as much as possible and generate a syntax tree even when encountering syntax errors. Relying on this mechanism, the token bag generator of MSCCD can sometimes still generate the correct token bags even when encountering syntax errors. When using the default error recovery strategy, the parser will first try to perform single-token insertion and single-token deletion to recover the parser process inline. If it cannot recover inline, the ANTLR parser will consume tokens until it finds a token that reasonably follows the current rule, which is called panic mode (Terence, 2013). When recovered by single-token insertion or deletion, the parser will report a "missing token" or "extraneous input" message. In this situation, MSCCD is able to produce the desired results. When it turns to panic mode, the ANTLR parser will throw NoViableAltException or InputMismatchException and report a "mismatched input" or "no viable alternative at input" message. If recovered successfully, MSCCD can also produce the desired results. If the recovery fails, the token bag cannot be extracted after the error position. Generally, automatic recovery for "mismatched input" is more likely to be successful than for "no viable alternative at input". This is because the InputMismatchException often involves more specific and localized issues, while the NoViableAltException may involve broader context and structural problems. In summary, successful inline recovery does not affect token bag generation; however, entering panic mode may lead to the loss of token bags due to the incompleteness of the parse tree, and NoViableAltException is more likely to cause an impact.

For JavaScript and C++, which generated a relatively higher number of syntactic analysis errors, we collected all their error messages and extracted the proportions of various message types (listed in Table 11). Firstly, in terms of the proportion of errors successfully recovered inline, C++'s 32.14% is higher than JavaScript's 18.05%. For the two exceptions that enter panic mode, C++ is mainly constituted by InputMismatchException, accounting for 59.11%, whereas JavaScript is predominantly due to NoViableAltException, accounting for 64.46%. This explains why JavaScript is more sensitive to syntax errors compared to C++, resulting in a greater loss of token bags.

We also collected the five most frequent error messages (listed in Table 12). As for the top frequent error messages, the second most common error message in JavaScript was "no viable alternative at input '{' ", which usually indicates an issue with brace matching. Braces in JavaScript are often used to define code blocks, such as function bodies. These elements appear more frequently at the top of the syntax tree, and problems occurring at higher positions typically mean more information loss. Conversely, the second most common error in C++, "missing ';' ", usually occurs at the end of an individual statement, is easy to handle, and does not lead to significant information loss.

6.1.3. Challenges in ANTLR grammar definitions

Grammars-v4 has grammar definition files for over 100 languages that can be used directly in MSCCD. However, these grammar description files could benefit from further refinement to improve their reliability. In experiments in Section 6.1.1, 23 out of 48 languages showed at least errors in syntactic analysis, even in ANTLR parser construction. Some of these issues can be attributed to problems in the ANTLR grammar definitions. For instance, parser generation failures or crashes during runtime can occur due to grammar definition files not adhering to the required rules. In addition, issues with the grammar definition file may also lead to incorrect results in syntax analysis, such as failing to scan the entire file (whether or not errors are reported). For users without expert-level knowledge, fixing these errors presents a significant challenge. Similarly, adding support for a new language requires creating ANTLR grammar definition files from scratch. Even if language definition files for the language already exist, porting them to ANTLR is not straightforward if those files are written in other grammars, such as LR grammar (left-to-right, rightmost derivation grammar), or formats such as ENF (Backus–Naur Form) and EBNF (Extended Backus–Naur Form). Therefore, the ability to extend MSCCD's grammar highly depends on the activity and contributions within the grammars-v4 community.

6.2. CCFinder series

The CCFinder series is also known for its multilingualism, as CCfinder and CCFinderX support 5–6 languages (Kamiya et al., 2002; Kamiya, 2021). Adding new support for them would require replacing the lexical or island-parser-like parser of the corresponding language. Therefore, they do not fit the definition of this study on a multilingual clone detector. Their successor, CCFinderSW, provides a multilingual processing module that converts specific parts of the grammar definition of the targeted language into regular expressions to build a source code processor for the target language automatically. In previous experiments, techniques of CCFinderSW were successfully applied to 43 of the 48 tested languages (Semura et al., 2018). In the current version, CCFinderSW has built-in support for 15 languages. Adding new language support to CCFinderSW is relatively easy, as a syntax description file need not be written. However, some languages whose grammar productions cannot be converted to regular expressions, such as Lua, do exist, and CCFinderSW cannot support such languages (Semura et al., 2018).

6.3. NiCad

The newest version of NiCad supports eight languages. The source code processing of NiCad is based on TXL, a programming language specifically designed for software analysis and source transformation (Roy and Cordy, 2008; Cordy, 2006). NiCad allows users to specify the analysis method for each language by TXL. However, similar to writing an ANTLR grammar for a language from the beginning, it is not easy to describe the analysis method using TXL for developers with limited knowledge. Besides, the current version of TXL supports 22 languages. Users can reuse TXL resources of these 22 languages to specify the code analysis method of NiCad.

6.4. Comparison

The current version of MSCCD supports the highest number of languages. Fig. 8 shows the relationship of the current built-in languages of the three multilingual code clone detectors, which allows the user to perform detection without any new configuration (including the configuration of syntax definitions). MSCCD supports the most significant number of languages, including 15 that were not included in the other two tools. With the resources of Grammars-v4, the addition of new language support to MSCCD and CCFinderSW is easy. Although MSCCD

Table 12
The 5 Most frequent error messages of JavaScript and C++.

Ranking	JavaScript		C++	
	Message	Percentages	Message	Percentages
1st	No viable alternative at input ':'	10%	Mismatched input '.'	29%
2nd	No viable alternative at input '{'	7%	Missing ';'	25%
3rd	No viable alternative at input ')'	6%	Mismatched input ')'	13%
4th	No viable alternative at input '('	6%	Mismatched input ':'	4%
5th	Extraneous input '<'	5%	Mismatched input ']'	4%

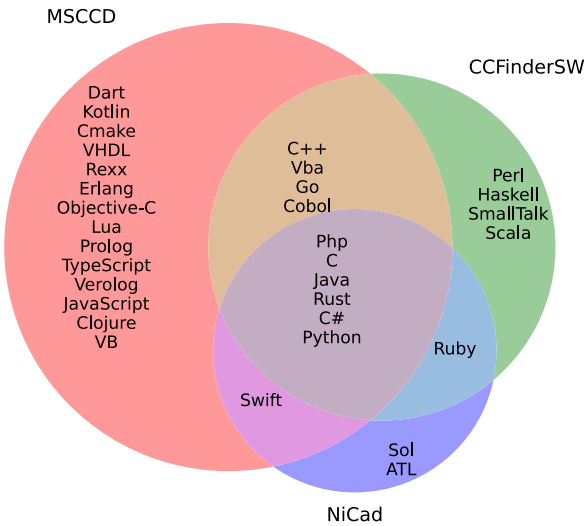

Fig. 8. Currently supported languages of three multilingual code clone detectors.

suffers from parser performance problems, it supports a much larger set of languages (25) than CCFinderSW (15) or NiCad (10). NiCad has the lowest built-in language support and resources available (22 languages the TXL community provides). In addition, NiCad encounters a large number of compiler errors during the experiments, which has a significant impact on clone detection.

> The answer to RQ3: MSCCD supports the maximum number of programming languages while extending CCFinderSW without existing syntax resources is more accessible. Furthermore, both MSCCD and NiCad are prone to syntax analysis failures.

7. Discussions

7.1. What is a better multilingual code clone detection approach?

As discussed in Section 6, the three main multilingual clone detectors had different language extensibility. Adding new language extensions to CCFinderSW was the easiest, but its approach could not be applied to some languages, such as Lua. Adding new language extensions to MSCCD by writing ANTLR syntax description files for a language was more complicated, but resource availability was the highest in this case. Adding new language extensions to NiCad required only writing partial grammar descriptions. However, it was still not easy to use, and its available resources were the lowest. The clone detection performances of the detectors were also different. Regarding the types of clones that can be detected, CCFinderSW supported up to T2, while MSCCD and NiCad could detect T3 clones. NiCad had a higher recall rate for near-miss clones than MSCCD, but its precision was less stable than MSCCD. Hence, no absolute leader among the three multilingual clone detectors was identified, but MSCCD was the most balanced.

Users are still required to weigh the pros and cons of using the best tool for their task requirements. In addition, there are more advanced or specialty, more prominent detection techniques, such as srcClone and NIL, that are yet to be extended to other languages (Nakagawa et al., 2021; Alomari and Stephan, 2022). Tools that use generalized tools for source code processing (MSCCD, NiCad) also suffer from parsing errors. To address such situations, we require the conception of a multilingual code clone detection infrastructure that can provide the easiest possible excuse and format to advance the use of official parsers, while providing a unified interface for clone detection without considering source code transformation. Developers can implement their clone detection methods on that infrastructure to support multiple languages.

7.2. Real-world applicability of MSCCD

This section mainly discusses the practical applications of MSCCD in the real world. Apart from general single-language clone detection, we attempt to outline the following application scenarios.

Scenario 1: Support for Multilingual Software Systems. As software systems increasingly utilize multiple languages, it is preferable to use a clone detector that supports all target languages simultaneously. Different code clone detection tools vary in precision, granularity, result sorting strategies, and output formats. Standardizing these elements would incur significant costs. Therefore, it is advisable to avoid mixing different clone detectors in the same task. MSCCD can be used to address this situation.

Scenario 2: Support for Many Higher-level Applications. Code clone detection is not the endpoint of a task but rather the starting point. Many tasks can be accomplished based on the results of code clone detection, such as vulnerable code detection (Kim et al., 2017), automatic software repair (Jiang et al., 2018), code search (Kim et al., 2018), and software refactoring support (Higo et al., 2004). When developing applications for these uses, MSCCD can be an instantly usable clone detector for the target language, playing a vital role in prototype validation and formal development. This allows developers to focus on developing higher-level functionalities without paying particular attention to implementing code clone detection technology, especially support for target languages.

MSCCD's most significant advantage lies in its ability to extend Type-3 level clone detection capabilities to many programming languages with minimal cost. The application scenarios listed above are based on this strength. Correspondingly, some of MSCCD's weaknesses also present challenges. Different tasks have varying requirements for the precision and expansiveness of clone detection tools. MSCCD cannot detect semantic clones and a part of Type-3 clones with very low similarity. It is not at the forefront regarding scalability. These weaknesses may render MSCCD unsuitable for specific tasks. Furthermore, most tasks require further development of higher-level programs based on MSCCD. The current version of MSCCD do not provide enough APIs to support the development of these programs, requiring developers to invest effort in it.

7.3. Threats to validity

7.3.1. Comparing with deep learning-based detectors

Clone detectors based on Deep Learning (DL) can accurately detect syntactic and semantic code clones. However, we did not include DL-based detectors in the scope of this research's experiments because they

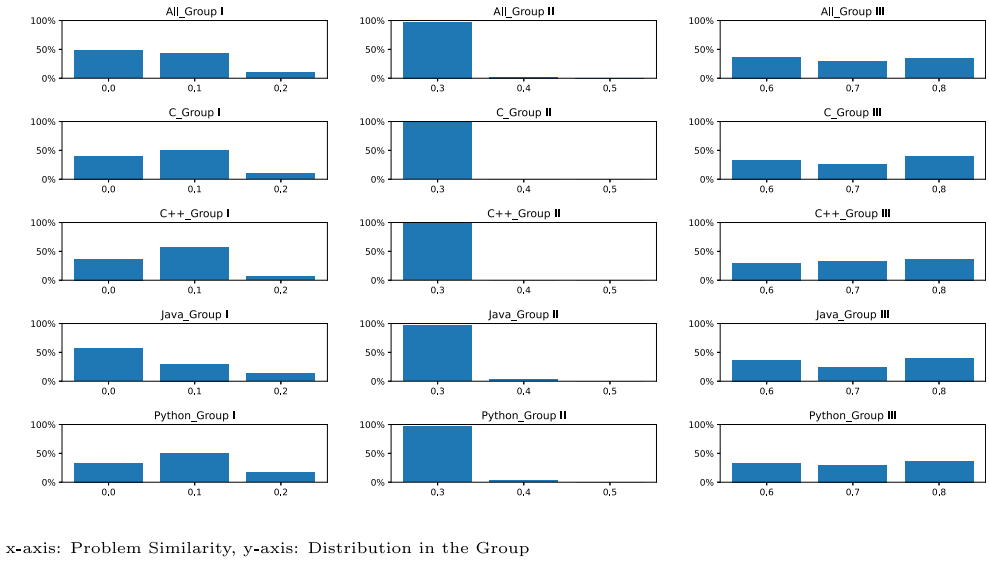

Fig. 9. Distribution of similarity of OJ problem pairs sampled.

lack generalizability, limiting their real-world usage. In the work of Choi et al. (2023), they evaluated three DL-based detectors, including CCLearner, ASTNN, and CodeBERT (Li et al., 2017; Zhang et al., 2019; Feng et al., 2020). They compared the accuracy of each detector when using the same and different benchmarks in training and testing. During their experiment, the *Matthews Correlation Coefficient* (*MCC*) score of all three detectors significantly decreased when using the different benchmarks for training and testing. E.g., CodeBERT performed a 0.998 *MCC* score when using CodeNet for training and testing. When using CodeNet for training and BigClongBench for testing, the *MCC* score decreased to 0.00432. For the syntactic code clones, their accuracy also decreases significantly. E.g., CodeBERT achieved a 0.881 *MCC* score when using BigCloneBench for training and testing. However, when using the CodeNet or Google Code Jam benchmark for training, the *MCC* scores for ST3 clones were 0.0039 and 0.0000778. Because the benchmarks in our work (BigCloneBench and CodeNet) are also used in their work, evaluating DL-based tools in our experiments cannot gain a different result in both detection accuracy and generalizability.

7.3.2. Configurations of the detectors to be compared

In Sections 4 and 5, we compared MSCCD with many state-of-the-art syntactic code clone detectors and used their default configurations as much as possible. However, according to the research of Ragkhitwetsagul et al. (2018), the default configurations may not be the best configurations of each clone detector. Not using the optimal configuration of each tool may reduce the effectiveness of the comparison. However, we still use the default configuration of each detector as much as possible for the following reasons. First, there is a lack of research on the optimal configuration of each detector, and it is hard to research the optimal configuration for all target tools in this work. Second, most existing studies use the default configuration for evaluation. Using the default configuration makes comparing the results with existing studies possible. In addition, the optimal configuration of each tool may contain different code lengths (such as the number of tokens or lines of code). Different code length configurations will cause each tool to detect different parts of the benchmark, resulting in unfair comparisons. In the experiments in Section 5, we configured the code length of each tool for the same size to ensure fairness.

7.3.3. Bias may introduced when constructing the multilingual precision benchmark

In constructing the multilingual precision benchmark, we extracted 30 pairs of OJ problems for each language, divided into three similarity intervals. The final precision of the target tool for a specific language within a given similarity interval is determined by its ability to differentiate the code from these 30 problem pairs correctly. The similarity interval represents the difficulty level. During this process, the average distribution of samples affects the effectiveness of this evaluation. Suppose the samples are concentrated on the right side of the similarity interval, meaning more question pairs are distributed on the higher difficulty side of the interval. In that case, the actual difficulty of this group is higher than the designated level, and the resulting accuracy might be lower than the actual level. Conversely, if the samples are concentrated on the left side, the actual difficulty of the group is lower, and the resulting accuracy might be higher.

We examined the distribution of the 12 sample groups in this study and created bar charts (Fig. 9) to illustrate the distribution of samples in each group. The x-axis of each subplot represents the similarity of question pairs, and each group was divided into three intervals of 10% for statistical purposes. The y-axis of each subplot shows the proportion of samples in that interval relative to the total number of samples in the group. Additionally, the first row of subplots represents the similarity distribution for all problem pairs. Firstly, the distribution characteristic of each sample group generally aligns with the overall characteristic, with no extreme cases observed. Groups I and III are relatively evenly distributed, whereas in Group II, samples with a similarity of 30%–40% account for over 99% of the total. This indicates that the actual difficulty of Group II is closer to a similarity of 30%–40% rather than 30%–60%, suggesting that Group 2 might be a slightly more lenient group.

The similarity between the description texts of OJ problems is used to represent the similarity between the problems themselves. A high similarity in description texts typically indicates a high similarity between the problems. However, exceptions may occur in the opposite scenario. Since the same meaning can sometimes be conveyed using different words and sentences in natural language, there is a possibility that the similarity between description texts is low while the actual similarity between the problems is high. If this occurs, a problem pair of higher actual difficulty might be placed in a group of lower difficulty, resulting in a higher difficulty level and a lower precision. Nevertheless, as each group comprises 30 problem pairs, the deviation of a single problem pair is unlikely to bias the overall group significantly. In our evaluation experiment, both Group I and Group II showed high overall precision. Therefore, it is highly probable that many higher-difficulty problems did not infiltrate the lower-difficulty groups.

We do not advocate using the benchmark created in this study as the sole version. We encourage users to employ the same methodology

Table 13
Description and possible solution of two similar problems from CodeNet.

Problem #2295	Title	Cross point
	Description	For given two segments s1 and s2, print the coordinate of the cross point of them. s1 is formed by end points p0 and p1, and s2 is formed by end points p2 and p3.
	Possible solution	**1. Check if the segments intersect.** 2. Output the coordinates of the cross
Problem #2296	Title	Distance
	Description	For given two segments s1 and s2, print the distance between them. s1 is formed by end points p0 and p1, and s2 is formed by end points p2 and p3.
	Possible solution	**1. Check if the segments intersect.** 2. Calculate the distance between a point from the one line segment to the other line segment if it does not intersect. 3. Use the above method to find the minimum distance.

to generate benchmarks for comparative validation using any database based on the OJ system. At this juncture, users should consider the abovementioned factors to avoid bias in certain groups. Users need to pay attention to the sample distribution of the problem sampling and consider checking the actual similarity of the problem pairs or employing alternative methods for problem similarity calculation.

7.3.4. Contentious in the treatment of clones from different problems

The problem groups drawn based on the similarity of the descriptive text posed challenges, especially for Group III, which had the highest text similarity. For some problem pairs, whether the codes from different problems should not be treated as a clone is controversial. Table 13 shows descriptions and possible solutions of two similar problems from CodeNet. Both of these problems involve operations on two given line segments, where problem #02295 is about finding the intersection coordinates of the two line segments, and problem #02296 deals with finding the shortest distance between them. A decomposition of the solutions to these problems reveals that they both include the sub-goal of "determining whether two line segments intersect". This means that although the overall objectives of the two problems are different, they share a common sub-objective. This similarity is also reflected in the submitted code: the code segments accomplishing the task of "determining whether two line segments intersect" are likely to have a high degree of similarity. For clone detection tools like MSCCD and SourcererCC, which have predetermined detection granularity and detect code clones between well-defined code blocks (or entire files), The actual comparison in this experiment is between the similarity of files, which are unaffected. However, tools like CCFinderSW, which do not have predetermined detection granularity and detect clones by attempting to match similar subsequences, are prone to false positives. In other words, the detector reports highly similar subsequences rather than similar file pairs. Some of these results are marked as clones by the benchmark's clone matcher because the file coverage exceeds 70%. From the perspective of these types of tools, their clone reports should not necessarily be considered false positives.

A potential solution is to construct a more fine-grained benchmark. In this study, our benchmark is file-level, encompassing a large number of program codes composed of multiple functions. If we were to use files written with a single function to build a function-level benchmark, it could circumvent the conflict issue described above. However, we are still determining whether sufficient single-function files exist in the original data to construct such a benchmark. At the same time, using only single-function files might introduce new biases due to changes in average length and other factors, which is also a matter of concern.

7.3.5. The other points for the benchmarks

The key idea of multilingual code clone detection evaluation is that all code pairs between accepted submissions from the same OJ problem can be considered code clones. Doing so means we do not need to label all the clone pairs manually. However, only code clones between files were tagged, while many programs from CodeNet consisted of multiple functions. This made it hard for our benchmark to evaluate block-level-only clone detectors because they do not report any file-level code clones. As mentioned above, constructing a function-level benchmark is helpful but challenging.

For the precision benchmark, since different languages may use different question sets even within the same group, the validity of the comparison between languages was reduced. We are considering reducing the benchmark size to use the same set of problems across different languages.

8. Related work

According to Roy's survey, a code clone detection approach usually has eight phases, among which the first three steps of preprocessing, transformation, and match detection are essential (Roy and Cordy, 2007).

The target code is partitioned and transformed while preprocessing and transforming to another intermediate representation that can calculate similarity. These phases require syntax information of the target language to remove unnecessary parts (comments and embedded code) and transformations. Therefore, most clone detectors only support a few languages (Zhu et al., 2022). To solve this problem, some researchers have proposed language processing mechanisms that allow clone detectors to support new languages more easily (Roy and Cordy, 2008; Semura et al., 2018; Zhu et al., 2022). Roy and Cordy proposed a flexible pretty-printing and code normalization approach based on TXL, a special-purpose programming language (Cordy, 2006), and implemented the NiCad clone detector (Roy and Cordy, 2008). Users are allowed to add new language support for NiCad by providing a source code analysis method written by TXL. Although writing TXL methods is more accessible than writing parser generator grammar, it is still difficult for users with limited knowledge. To date, NiCad does not support a number of languages. CCFinderSW (Semura et al., 2018) implements a multilingual lexer to generate a token sequence for clone detection (Kamiya, 2021). The main idea is to convert the grammar rules of comment, identifier, keyword, and literal into regular expressions. Since there are only four syntax rules to focus on, adding language support to CCFinderSW is the easiest. However, some syntax rules cannot be converted to regular languages, such as the comment of Lua; thus, it is not supported by CCFinderSW. In addition, the

approach of CCFinderSW can only generate token sequences. Currently, it only supports Type-2 clone detection, which is extremely difficult to extend in the future. The MSCCD source code processing approach can generate token bags and can be simply extended to token sequences and PTs. More accurate clone detection techniques are expected to be implemented using MSCCD.

For match detection, the clone detector compares partitioned code to find similar code segments. These detection approaches can be categorized by the level of intermediate representation as text-based (Roy and Cordy, 2008; Nasehi et al., 2007), token-based (Sajnani et al., 2016; Zhu et al., 2022; Wang et al., 2018; Kamiya et al., 2002; Basit and Jarzabek, 2007; Ragkhitwetsagul and Krinke, 2019), tree-based (Jiang et al., 2007; Baxter et al., 1998; Koschke et al., 2006), metrics-based (Saini et al., 2018), and graph-based (Wang et al., 2017; Kamalpriya and Singh, 2017). NiCad (Roy and Cordy, 2008), iClones (Göde and Koschke, 2009), Deckard (Jiang et al., 2007), SourcererCC (Sajnani et al., 2016), Oreo (Saini et al., 2018), and CloneWorks (Svajlenko and Roy, 2017) are some of the well-known Type-3 level clone detectors. Some detectors focus on individual metrics, such as NIL (Nakagawa et al., 2021) and CCAligner (Wang et al., 2018) that focus on detecting large-variance and large-gap clones, while SAGA (Li et al., 2020) extends the scalability of clone detection to 550MLOC. Although there is a gap between MSCCD and the latest detectors in terms of near-miss clone detection and scalability, it supports the largest number of programming languages and also has above average detection accuracy and scalability. This makes MSCCD the most balanced clone detector. We expect more clone detection techniques to have multi-language implementations in the future.

All the aforementioned tools are designed mainly for syntactic code clones, and there is another class of techniques designed to detect semantic code. SrcClone analyzes semantic similarity by program slicing technique (Alomari and Stephan, 2022). Some AI-based clone detectors are also frequently used to detect semantic code clones (Wu et al., 2020; Yu et al., 2023; Fang et al., 2020). SCDetector performs well by combining the information about the token and the control flow graphs using a neural network (Wu et al., 2020). Yu et al. proposed an approach to learn semantics based on CFG (Control Flow Graph) or PDG (Program Dependency Graph), while Fang et al. proposed to learn semantic by applying fusion embedding techniques. However, according to research from Choi et al. (2023) and Liu et al. (2021), AI-based clone detection tools have limitations in their generalization capabilities. Therefore, we did not include these AI-based approaches in our experiments. The existing multilingual code clone detectors (MSCCD, NiCad, CCFinderSW) cannot detect semantic clones. Besides, neither code slicing nor control flow graph generation has a multilinguistic approach. Therefore, multilingual semantic clone detection is not supported at the moment.

Unlike the above-introduced approaches, cross-language code clone detectors aim to find code clones between different programming languages. Perez et al. proposed an approach to detect code clones between Java and Python that learn token-level vector representations by an LSTM-based neural network (Perez and Chiba, 2019a). LICCA can detect clones between five languages by generating a high-level representation of code from the SSQSA platform (Vislavski et al., 2018). CLCDSA can detect cross-languages without creating other intermediate representations (Nafi et al., 2019). It is implemented by learning and comparing the similarity of features. Currently, there are no multilingual cross-language clone detectors. However, the source code processing approach of MSCCD can support multiple intermediate representations of code, such as PT. Once a PT-based cross-language clone detection approach is proposed, it can be easily added to a multilingual implementation.

Traditional and cross-language code clones assume that the targeted software is developed in a single language. However, a large amount of software is developed in multiple languages for compatibility with older systems or the need to take advantage of different features of different languages simultaneously. These include many source code files written in multiple languages (the most typical ones are HTML-JavaScript and Java server pages) (Rajapakse and Jarzabek, 2007; Muhammad et al., 2013). A clone pair can be defined such that both sides are composed of multiple languages as a language-mixed code clone. Nakamura et al. proposed an approach to detect language-mixed code clones between web systems (Nakamura et al., 2016). Besides, some frequently used language groups are supported, such as Python and Ruby. One of our future challenges is to extend MSCCD for language-mixed code clone detection using a technique such as the island grammars.

An enduring problem for benchmarking code clone detectors for recall and precision exists. Since the development of such a benchmark requires marking all correct code clones in all code pairs, there are no automated methods of marking code clones (perfect clone detector), and the workload for manually checking code clones is far beyond the limit. Researchers initially evaluated code clone detectors by comparing their targeting at large-scale open-source software, such as Linux and NetBSD (Jiang et al., 2007; Kamiya et al., 2002; Li et al., 2006). Bellon et al. created the first large-scale benchmark to visually judge whether a code clone was detected (Bellon et al., 2007). The most frequently used benchmark is BigCloneBench (Svajlenko and Roy, 2015, 2016) created by Scajlenko and Roy. BigCloneBench contains over 7,800,000 pairs of Java clones and can report recalls in each of the six categories: T1, T2, VST3, ST3, MT3, and WT3/T4. Svajlenko and Roy also published a mutation and injection framework to evaluate the recall of clone detectors for C (Svajlenko and Roy, 2021). Some studies have attempted to advance the evaluation of precision (Farmahinifarahani et al., 2019; Saini et al., 2019). However, current precision evaluations still require random sampling and manual judgment. Al-Omari et al. proposed SemanticCloneBench (Al-Omari et al., 2020), which collects semantic code clones from Stack Overflow answers. SemanticCloneBench supports Java, C, C#, and Python. Alam et al. proposed GPTCloneBench, which is based on SemanticCloneBench and GPT-3 model (Alam et al., 2023). GPTCloneBench provides more functional code examples in higher quality. However, these benchmarks were not designed for evaluating syntactic code clone detectors.

In Section 5 of this study, we created a benchmark to evaluate recall and precision automatically for four languages by using the project CodeNet,[16] which included more than 13,000,000 submissions from AIZU Online Judge and AtCoder. Competitive programming codes have always been used for evaluating deep-learning-based code clone approaches (Zhang et al., 2019; Zhao and Huang, 2018; Perez and Chiba, 2019b); however, our study is the first to use such an approach for syntactic clone detection evaluation.

9. Conclusions and future work

This work addressed the need for existing multilingual code clone detectors. We proposed a code block extraction method based on the ANTLR parser generator and implemented it into MSCCD. MSCCD can support the most popular programming languages and detect T3 clones. Furthermore, we constructed the first multilingual syntactic code clone detection evaluation benchmark, which measures precision and recall for C, C++, Java, and Python. According to the results of evaluation experiments, MSCCD supports the most languages among the existing tools and is at the same level of detection performance as the existing tools. Although MSCCD is already the most balanced multilingual clone detector, it still needs help with problems like parser errors. An intermediate layer for multilingual clone detection can be implemented in the future to promote the use of official parsers and to enable various detection techniques to be implemented directly in more languages.

In the future, we plan to continue improving the performance of the evaluation benchmark for multilingual clone detection and explore the best implementation of the middle layer.

[16] https://github.com/IBM/Project_CodeNet.

Declaration of competing interest

The authors declare the following financial interests/personal relationships which may be considered as potential competing interests: Yoshiki Higo, Katsuro Inoue, Takashi Ishio, Yuki Manabe, Raula Kula, Kozo Okano, Kenji Fujiwara, Koji Toda, Masanari Kondo, Hajimu Iida, Hironori Washizaki.

Acknowledgments

This work was supported by JST, Japan, PRESTO Grant Number JPMJPR21PA, Japan. Also, this work was supported by JSPS KAKENHI, Japan Grant Number JP20K11745, JP22K11975, JP23K11046 and JP24K02923.

References

Ain, Q.U., Butt, W.H., Anwar, M.W., Azam, F., Maqbool, B., 2019. A systematic review on code clone detection. IEEE Access 7, 86121–86144. http://dx.doi.org/10.1109/access.2019.2918202.

Al-Omari, F., Roy, C.K., Chen, T., 2020. Semanticclonebench: A semantic code clone benchmark using crowd-source knowledge. In: 2020 IEEE 14th International Workshop on Software Clones. IWSC, IEEE, pp. 57–63.

Alam, A.I., Roy, P.R., Al-Omari, F., Roy, C.K., Roy, B., Schneider, K.A., 2023. GPT-CloneBench: A comprehensive benchmark of semantic clones and cross-language clones using GPT-3 model and SemanticCloneBench. In: 2023 IEEE International Conference on Software Maintenance and Evolution. ICSME, pp. 1–13. http://dx.doi.org/10.1109/ICSME58846.2023.00013.

Alomari, H.W., Stephan, M., 2022. Clone detection through srcclone: A program slicing based approach. J. Syst. Softw. 184, 111115. http://dx.doi.org/10.1016/j.jss.2021.111115, URL https://www.sciencedirect.com/science/article/pii/S0164121221002120.

Basit, H.A., Jarzabek, S., 2007. Efficient token based clone detection with flexible tokenization. In: Proceedings of the the 6th Joint Meeting of the European Software Engineering Conference and the ACM SIGSOFT Symposium on the Foundations of Software Engineering. pp. 513–516.

Baxter, I.D., Yahin, A., Moura, L., Sant'Anna, M., Bier, L., 1998. Clone detection using abstract syntax trees. In: Proceedings of International Conference on Software Maintenance. IEEE, pp. 368–377.

Bellon, S., Koschke, R., Antoniol, G., Krinke, J., Merlo, E., 2007. Comparison and evaluation of clone detection tools. IEEE Trans. Softw. Eng. 33 (9), 577–591.

Cheung, W.T., Ryu, S., Kim, S., 2016. Development nature matters: An empirical study of code clones in JavaScript applications. Empir. Softw. Eng. 21 (2), 517–564.

Choi, E., Fuke, N., Fujiwara, Y., Yoshida, N., Inoue, K., 2023. Investigating the generalizability of deep learning-based clone detectors. In: 2023 IEEE/ACM 31st International Conference on Program Comprehension. ICPC, pp. 181–185. http://dx.doi.org/10.1109/ICPC58990.2023.00032.

Cordy, J.R., 2006. The TXL source transformation language. Sci. Comput. Program. 61 (3), 190–210.

Danial, A., 2021. Cloc: v1.92. http://dx.doi.org/10.5281/zenodo.5760077.

Ducasse, S., Rieger, M., Demeyer, S., 1999. A language independent approach for detecting duplicated code. In: Proceedings IEEE International Conference on Software Maintenance-1999 (ICSM'99).'software Maintenance for Business Change'(Cat. No. 99CB36360). IEEE, pp. 109–118.

Fang, C., Liu, Z., Shi, Y., Huang, J., Shi, Q., 2020. Functional code clone detection with syntax and semantics fusion learning. In: Proceedings of the 29th ACM SIGSOFT International Symposium on Software Testing and Analysis. In: ISSTA 2020, Association for Computing Machinery, New York, NY, USA, pp. 516–527. http://dx.doi.org/10.1145/3395363.3397362.

Farmahinifarahani, F., Saini, V., Yang, D., Sajnani, H., Lopes, C.V., 2019. On precision of code clone detection tools. In: 2019 IEEE 26th International Conference on Software Analysis, Evolution and Reengineering. SANER, pp. 84–94.

Feng, Z., Guo, D., Tang, D., Duan, N., Feng, X., Gong, M., Shou, L., Qin, B., Liu, T., Jiang, D., et al., 2020. Codebert: A pre-trained model for programming and natural languages. arXiv preprint arXiv:2002.08155.

Fowler, M., 1999. Refactoring: Improving the Design of Existing Code. Addison-Wesley, Boston, MA, USA.

Göde, N., Koschke, R., 2009. Incremental clone detection. In: 2009 13th European Conference on Software Maintenance and Reengineering. IEEE, pp. 219–228.

Group, A., et al., 2006. Ijadataset 2.0.

Higo, Y., Kamiya, T., Kusumoto, S., Inoue, K., 2004. Refactoring support based on code clone analysis. In: International Conference on Product Focused Software Process Improvement. Springer, pp. 220–233.

Jiang, L., Misherghi, G., Su, Z., Glondu, S., 2007. Deckard: Scalable and accurate tree-based detection of code clones. In: Proceedings of the 29th International Conference on Software Engineering. pp. 96–105.

Jiang, J., Xiong, Y., Zhang, H., Gao, Q., Chen, X., 2018. Shaping program repair space with existing patches and similar code. In: Proceedings of the 27th ACM SIGSOFT International Symposium on Software Testing and Analysis. In: ISSTA 2018, Association for Computing Machinery, New York, NY, USA, pp. 298–309. http://dx.doi.org/10.1145/3213846.3213871.

Kamalpriya, C., Singh, P., 2017. Enhancing program dependency graph based clone detection using approximate subgraph matching. In: 2017 IEEE 11th International Workshop on Software Clones. IWSC, IEEE, pp. 1–7.

Kamiya, T., 2021. CCFinderX: An interactive code clone analysis environment. In: Code Clone Analysis. Springer, pp. 31–44.

Kamiya, T., Kusumoto, S., Inoue, K., 2002. CCFinder: A multilinguistic token-based code clone detection system for large scale source code. IEEE Trans. Softw. Eng. 28 (7), 654–670.

Kim, K., Kim, D., Bissyandé, T.F., Choi, E., Li, L., Klein, J., Traon, Y.L., 2018. FaCoY: a code-to-code search engine. In: Proceedings of the 40th International Conference on Software Engineering. pp. 946–957.

Kim, S., Woo, S., Lee, H., Oh, H., 2017. Vuddy: A scalable approach for vulnerable code clone discovery. In: 2017 IEEE Symposium on Security and Privacy. SP, IEEE, pp. 595–614.

Koschke, R., Falke, R., Frenzel, P., 2006. Clone detection using abstract syntax suffix trees. In: 2006 13th Working Conference on Reverse Engineering. IEEE, pp. 253–262.

Li, L., Feng, H., Zhuang, W., Meng, N., Ryder, B., 2017. CCLEARNER: A deep learning-based clone detection approach. In: Proceedings of International Conference on Software Maintenance and Evolution. pp. 249–260.

Li, Z., Lu, S., Myagmar, S., Zhou, Y., 2006. CP-miner: finding copy-paste and related bugs in large-scale software code. IEEE Trans. Softw. Eng. 32 (3), 176–192. http://dx.doi.org/10.1109/TSE.2006.28.

Li, G., Wu, Y., Roy, C.K., Sun, J., Peng, X., Zhan, N., Hu, B., Ma, J., 2020. SAGA: efficient and large-scale detection of near-miss clones with GPU acceleration. In: Proceedings of the 27th International Conference on Software Analysis, Evolution and Reengineering. pp. 272–283.

Liu, C., Lin, Z., Lou, J.-G., Wen, L., Zhang, D., 2021. Can neural clone detection generalize to unseen functionalities. In: 2021 36th IEEE/ACM International Conference on Automated Software Engineering. ASE, pp. 617–629. http://dx.doi.org/10.1109/ASE51524.2021.9678907.

Lu, S., Guo, D., Ren, S., Huang, J., Svyatkovskiy, A., Blanco, A., Clement, C.B., Drain, D., Jiang, D., Tang, D., Li, G., Zhou, L., Shou, L., Zhou, L., Tufano, M., Gong, M., Zhou, M., Duan, N., Sundaresan, N., Deng, S.K., Fu, S., Liu, S., 2021. CodeXGLUE: A machine learning benchmark dataset for code understanding and generation. CoRR abs/2102.04664.

Muhammad, T., Zibran, M.F., Yamamoto, Y., Roy, C.K., 2013. Near-miss clone patterns in web applications: An empirical study with industrial systems. In: Proceedings of the 26th IEEE Canadian Conference on Electrical and Computer Engineering. pp. 1–6.

Nafi, K.W., Kar, T.S., Roy, B., Roy, C.K., Schneider, K.A., 2019. CLCDSA: Cross language code clone detection using syntactical features and API documentation. In: Proceedings of the 34th International Conference on Automated Software Engineering. pp. 1026–1037.

Nakagawa, T., Higo, Y., Kusumoto, S., 2021. NIL: Large-scale detection of large-variance clones. In: Proceedings of the 29th ACM Joint Meeting on European Software Engineering Conference and Symposium on the Foundations of Software Engineering. In: ESEC/FSE 2021, Association for Computing Machinery, New York, NY, USA, pp. 830–841. http://dx.doi.org/10.1145/3468264.3468564.

Nakamura, Y., Choi, E., Yoshida, N., Haruna, S., Inoue, K., 2016. Towards detection and analysis of interlanguage clones for multilingual web applications. In: Proceedings of the 23rd International Conference on Software Analysis, Evolution, and Reengineering. Vol. 3, pp. 17–18.

Nasehi, S.M., Sotudeh, G.R., Gomrokchi, M., 2007. Source code enhancement using reduction of duplicated code. In: Proceedings of the 25th Conference on IASTED International Multi-Conference: Software Engineering. pp. 192–197.

Navarro, G., 2001. A guided tour to approximate string matching. ACM Comput. Surv. 33 (1), 31–88. http://dx.doi.org/10.1145/375360.375365.

Nishi, M.A., Damevski, K., 2018. Scalable code clone detection and search based on adaptive prefix filtering. J. Syst. Softw. 137, 130–142.

Parr, T.J., Quong, R.W., 1995. ANTLR: A predicated-LL (k) parser generator. Softw. - Pract. Exp. 25 (7), 789–810.

Perez, D., Chiba, S., 2019a. Cross-language clone detection by learning over abstract syntax trees. In: Proceedings of the 16th International Conference on Mining Software Repositories. pp. 518–528.

Perez, D., Chiba, S., 2019b. Cross-language clone detection by learning over abstract syntax trees. In: 2019 IEEE/ACM 16th International Conference on Mining Software Repositories. MSR, pp. 518–528. http://dx.doi.org/10.1109/MSR.2019.00078.

Pizzolotto, D., Matsushita, M., Inoue, K., 2022. Code clone detection in rust intermediate representation.

Puri, R., Kung, D.S., Janssen, G., Zhang, W., Domeniconi, G., Zolotov, V., Dolby, J., Chen, J., Choudhury, M., Decker, L., Thost, V., Buratti, L., Pujar, S., Ramji, S., Finkler, U., Malaika, S., Reiss, F., 2021. CodeNet: A large-scale AI for code dataset for learning a diversity of coding tasks. http://dx.doi.org/10.48550/ARXIV.2105.12655, URL https://arxiv.org/abs/2105.12655.

Ragkhitwetsagul, C., Krinke, J., 2019. Siamese: scalable and incremental code clone search via multiple code representations. Empir. Softw. Eng. 24 (4), 2236–2284.

Ragkhitwetsagul, C., Krinke, J., Clark, D., 2018. A comparison of code similarity analysers. Empir. Softw. Eng. 23, 2464–2519.

Rajapakse, D.C., Jarzabek, S., 2007. Using server pages to unify clones in web applications: A trade-off analysis. In: Proceedings of the 29th International Conference on Software Engineering. pp. 116–126.

Roy, C.K., Cordy, J.R., 2007. A survey on software clone detection research. Queen's Sch. Comput. TR 541 (115), 64–68.

Roy, C.K., Cordy, J.R., 2008. NiCad: Accurate detection of near-miss intentional clones using flexible pretty-printing and code normalization. In: Proceedings of the 16th International Conference on Program Comprehension. pp. 172–181. http://dx.doi.org/10.1109/ICPC.2008.41.

Saini, V., Farmahinifarahani, F., Lu, Y., Baldi, P., Lopes, C.V., 2018. Oreo: Detection of clones in the twilight zone. In: Proceedings of the 2018 26th ACM Joint Meeting on European Software Engineering Conference and Symposium on the Foundations of Software Engineering. pp. 354–365.

Saini, V., Farmahinifarahani, F., Lu, Y., Yang, D., Martins, P., Sajnani, H., Baldi, P., Lopes, C.V., 2019. Towards automating precision studies of clone detectors. In: 2019 IEEE/ACM 41st International Conference on Software Engineering. ICSE, IEEE, pp. 49–59.

Sajnani, H., Saini, V., Svajlenko, J., Roy, C.K., Lopes, C.V., 2016. SourcererCC: Scaling code clone detection to big-code. In: Proceedings of the 38th International Conference on Software Engineering. pp. 1157–1168. http://dx.doi.org/10.1145/2884781.2884877.

Semura, Y., Yoshida, N., Choi, E., Inoue, K., 2018. Multilingual detection of code clones using ANTLR grammar definitions. In: 2018 25th Asia-Pacific Software Engineering Conference. APSEC, pp. 673–677. http://dx.doi.org/10.1109/apsec.2018.00088.

Stefik, A., Hanenberg, S., 2014. The programming language wars: Questions and responsibilities for the programming language community. In: Proceedings of the 2014 ACM International Symposium on New Ideas, New Paradigms, and Reflections on Programming & Software. In: Onward! 2014, Association for Computing Machinery, New York, NY, USA, pp. 283–299. http://dx.doi.org/10.1145/2661136.2661156.

Svajlenko, J., Roy, C.K., 2015. Evaluating clone detection tools with BigCloneBench. In: Proceedings of the 31st International Conference on Software Maintenance and Evolution. pp. 131–140. http://dx.doi.org/10.1109/icsm.2015.7332459.

Svajlenko, J., Roy, C.K., 2016. BigCloneEval: A clone detection tool evaluation framework with BigCloneBench. In: 2016 IEEE International Conference on Software Maintenance and Evolution. ICSME, pp. 596–600.

Svajlenko, J., Roy, C.K., 2017. Fast and flexible large-scale clone detection with CloneWorks. In: Proceedings of the 39th International Conference on Software Engineering Companion. pp. 27–30.

Svajlenko, J., Roy, C.K., 2021. The mutation and injection framework: Evaluating clone detection tools with mutation analysis. IEEE Trans. Softw. Eng. 47 (5), 1060–1087.

Terence, P., 2013. The Definitive ANTLR 4 Reference. Pragmatic Bookshelf.

Urma, R.-G., 2017. Programming Language Evolution. Tech. Rep. UCAM-CL-TR-902, University of Cambridge, Computer Laboratory, http://dx.doi.org/10.48456/tr-902, URL https://www.cl.cam.ac.uk/techreports/UCAM-CL-TR-902.pdf.

Vislavski, T., Rakić, G., Cardozo, N., Budimac, Z., 2018. LICCA: A tool for cross-language clone detection. In: 2018 IEEE 25th International Conference on Software Analysis, Evolution and Reengineering. SANER, pp. 512–516. http://dx.doi.org/10.1109/SANER.2018.8330250.

Wang, W., Deng, Z., Xue, Y., Xu, Y., 2023. Ccstokener: Fast yet accurate code clone detection with semantic token. J. Syst. Softw. 111618.

Wang, P., Svajlenko, J., Wu, Y., Xu, Y., Roy, C.K., 2018. CCAligner: a token based large-gap clone detector. In: Proceedings of the 40th International Conference on Software Engineering. pp. 1066–1077.

Wang, M., Wang, P., Xu, Y., 2017. CCSharp: An efficient three-phase code clone detector using modified PDGs. In: 2017 24th Asia-Pacific Software Engineering Conference. APSEC, IEEE, pp. 100–109.

Wu, Y., Zou, D., Dou, S., Yang, S., Yang, W., Cheng, F., Liang, H., Jin, H., 2020. SCDetector: software functional clone detection based on semantic tokens analysis. In: Proceedings of the 35th IEEE/ACM International Conference on Automated Software Engineering. pp. 821–833.

Yu, D., Yang, Q., Chen, X., Chen, J., Xu, Y., 2023. Graph-based code semantics learning for efficient semantic code clone detection. Inf. Softw. Technol. 156, 107130. http://dx.doi.org/10.1016/j.infsof.2022.107130, URL https://www.sciencedirect.com/science/article/pii/S0950584922002397.

Zhang, J., Wang, X., Zhang, H., Sun, H., Wang, K., Liu, X., 2019. A novel neural source code representation based on abstract syntax tree. In: Proceedings of the 41st International Conference on Software Engineering. pp. 783–794.

Zhao, G., Huang, J., 2018. DeepSim: Deep learning code functional similarity. In: Proceedings of the 26th ACM Joint Meeting on European Software Engineering Conference and Symposium on the Foundations of Software Engineering. pp. 141–151.

Zhu, W., Yoshida, N., Kamiya, T., Choi, E., Takada, H., 2022. MSCCD: Grammar pluggable clone detection based on ANTLR parser generation. In: Proceedings of the 30th IEEE/ACM International Conference on Program Comprehension. ICPC '22, Association for Computing Machinery, New York, NY, USA, pp. 460–470. http://dx.doi.org/10.1145/3524610.3529161.

Wenqing Zhu is a Ph.D. student at the Graduate School of Informatics, Nagoya University. He received his Master's degree from Nagoya University in 2022 and his B.E. from Northeast Normal University. He is interested in program analysis.

Norihiro Yoshida is a Professor at Ritsumeikan University. He received his Ph.D. from Osaka University in 2009. Before joining Ritsumeikan University, he was an associate professor in the Graduate School of Informatics at Nagoya University from 2014 to 2022 and an assistant professor in the Graduate School of Information Science at the Nara Institute of Science and Technology from 2010 to 2014. His research interests include software maintenance and testing.

Toshihiro Kamiya received his M.Eng. in 1999 and Ph.D.Eng. in 2001 from the Department of Informatics and Mathematical Science at Osaka University. From 2001 to 2004, he served as a Research Associate in the Intelligent Cooperation and Control Group at RPESTO, a part of the Japan Science and Technology Corporation. He then worked as a Researcher at the National Institute of Advanced Industrial Science and Technology from 2005 to 2009. He held the position of Associate Professor at Future University Hakodate from 2010 to 2014. Currently, he is a Professor at Shimane University. His research interests include program comprehension, as well as static and dynamic program analysis. He is a member of IPSJ and IEEE.

Eunjong Choi is an associate professor in the Faculty of Information and Human Sciences at the Kyoto Institute of Technology, Japan since March 2024. Before that, she was an assistant professor at the same institution from March 2019 to February 2024. She previously held assistant Professor positions at Nara Institute of Science and Technology from April 2016 to March 2019, and at Osaka School of International Public Policy, Osaka University from April 2015 to March 2016. Her research interests are in the areas of software engineering, particularly focusing on code clone management, and refactoring support.

Hiroaki Takada is a professor at the Institutes of Innovation for Future Society, Nagoya University. He is also a professor and the Executive Director of the Center for Embedded Computing Systems (NCES), the Graduate School of Informatics, Nagoya University. He received his Ph.D. in Information Science from the University of Tokyo in 1996. He was a Research Associate at the University of Tokyo from 1989 to 1997 and was a Lecturer and then an Associate Professor at Toyohashi University of Technology from 1997 to 2003. His research interests include real-time operating systems, real-time scheduling theory, and automotive embedded systems. He is a fellow of IPSJ and JSSST and is a member of ACM, IEEE, IEICE, and JSAE.